Does Forgetting Help? Studying Context Reset Policies in LLM Fault Localization

Majid Babaei[a,*]

[a]School of Computing, University of the Fraser Valley,

ARTICLE INFO

Keywords:
Fault localization
Large language models
Context management
Models at runtime
Empirical software engineering
Reproducibility

ABSTRACT

Large language model (LLM) agents that localize faults from bug reports accumulate a dialogue in which checked facts, tentative explanations and the reasoning that links them are interleaved. When such an investigation stalls, a common remedy is to clear the context and start afresh, yet a reset that discards the dialogue can also discard what the agent has already established. We ask whether forgetting helps a localization agent, and what it should forget. We present MORPH-FL, a fault-localization framework for Python that keeps observations, hypotheses and dialogue in separate runtime models, so that a reset can clear dialogue and prune hypotheses while preserving every recorded observation. Using the framework as an experimental instrument, we compare four context policies on 50 SWE-bench issues from 12 repositories with two open 7B-scale code LLMs: adaptive reset on stagnation, continuous context, fixed-interval reset, and an observation-only view that shows the investigator its observations but neither its hypotheses nor its dialogue. Resetting the context, adaptively or on a schedule, did not change which issues were localized at rank one. The observation-only view localized more issues with one model and, with the other, kept grounded hypotheses on nearly every issue where the richer views kept almost none, at a higher time cost. Because rankings fall back to lexical retrieval when an investigation yields nothing, equal accuracy concealed very different investigations. The results suggest that what an agent is shown matters more than when its context is reset, and that context policies must be judged on investigation output and cost as well as accuracy.

1. Introduction

Anyone who has debugged a hard problem knows what it is like to be trapped by one's own reasoning. An early hunch decides which code gets read, which checks get run and how an ambiguous result is interpreted, and before long the investigation is circling. Stepping away and coming back with fresh eyes often helps. It helps, however, only because the developer keeps what was learned along the way, e.g., which files were ruled out. Effective forgetting is selective: *it discards outdated assumptions while preserving relevant knowledge*.

LLM agents now carry out the same kind of investigation. Given a bug report and a repository, systems such as AutoFL, SoapFL and Agentless navigate the code, propose explanations and narrow the candidates until they can name the locations a developer should inspect (Kang et al., 2024; Qin et al., 2025; Xia et al., 2025). As the investigation proceeds, its context grows into a single transcript in which source facts, proposed fault mechanisms and the reasoning that connects them are interleaved. LLMs do not use such contexts reliably. Information in the middle of a long input is overlooked, irrelevant material distracts, and a model that takes a wrong turn early in a multi-turn conversation seldom recovers (Liu et al., 2024; Shi et al., 2023; Laban et al., 2026; Hong et al., 2025). Agent designers respond by truncating, summarizing, masking or restarting the context (Packer et al., 2024; Yang et al., 2024b; Lindenbauer et al., 2025; Tan et al., 2026).

These remedies treat context as text to be shortened, and for fault localization that view misses what matters. A transcript contains information with different useful lifetimes. The observation that a given line of a fixed source revision contains a given expression remains true for the rest of the investigation. A hypothesis about why the failure occurs is a conjecture, and it can anchor the agent long after it has stopped being useful. Truncation and summarization cannot tell the two apart. A reset intended to escape a bad hypothesis may erase the refutation that had ruled it out, and a summary may turn a conjecture into something that reads like a fact. Whether forgetting helps a localization agent therefore depends on *what* is forgotten and *when*. To our knowledge, this question has not been studied.

*Corresponding author.
majid.babaei@ufv.ca (M. Babaei)
ORCID(s): 0000-0002-1394-4030 (M. Babaei)

We study it with MORPH-FL, a fault-localization framework for Python repositories. Drawing on models at runtime (Blair et al., 2009) and on hypothesis-driven debugging (Zeller, 2009), MORPH-FL keeps the state of an investigation in four linked runtime models that describe the reported behavior, the source structure, the competing hypotheses and the evidence, and it builds every LLM request as a view of these models. Because facts and conjectures are distinct model elements, retention can be specified per kind of knowledge. When a stagnation monitor detects that the investigation has stopped producing new observations, MORPH-FL clears the investigators' dialogues and prunes the hypotheses to a small portfolio chosen for score and diversity, while every observation and the cumulative budget survive. Keeping state outside the dialogue is not new in itself; what MORPH-FL contributes is a precise retention specification and an implementation of it. The same machinery also lets us change what a request shows while holding everything else fixed, which makes MORPH-FL an instrument for comparing context policies as well as a localization approach.

We use this instrument to compare four context policies that span the space between never forgetting and forgetting everything except the facts. *Continuous context* keeps all dialogue and hypotheses. *Adaptive reset*, the full MORPH-FL design, resets when the monitor detects stagnation. *Fixed reset* applies the same reset on a fixed schedule. The *observation-only view* shows the investigator the report, source excerpts and recorded observations, but neither its earlier hypotheses nor its dialogue. We run every policy on the same 50 issues, drawn from twelve repositories of SWE-bench Lite and Verified (Jimenez et al., 2024; Chowdhury et al., 2024), with two open code LLMs, Qwen2.5-Coder-7B and DeepSeek-Coder-6.7B. The study comprises all 400 model–workflow–issue investigations and 100 deterministic retrieval controls, and it is guided by four research questions:

RQ1 (Ranking) How do the context policies compare in how well they rank the files and functions changed by the fix, with each model and against retrieval alone?

RQ2 (Reset exposure) How often do adaptive and fixed resets actually occur, and do investigations that can reset localize more issues than continuous context?

RQ3 (Resources) What token and time costs accompany each policy under a common budget?

RQ4 (Investigation output) What do the final hypothesis pools and response diagnostics reveal that ranking accuracy does not?

The results did not confirm the design we set out to evaluate. Neither adaptive nor fixed resets localized a single additional issue at rank one: with both models, the two reset policies and continuous context hit exactly the same issues. The policy that forgot the most did best. With Qwen, the observation-only view raised file Top-1 accuracy from 42% to 50%; with DeepSeek, it left accuracy unchanged but ended with grounded hypotheses on 47 of 50 issues, where the richer views ended with them on six or seven. This second result exposes a measurement problem that goes beyond our system. When no hypothesis survives, MORPH-FL falls back to lexical retrieval order, so the four DeepSeek workflows reached the same accuracy as retrieval alone while doing very different amounts of investigative work. The observation-only view was also 1.5 to 2 times slower. We treat it as a promising configuration to confirm rather than as a validated treatment, because we singled it out after seeing outcomes, it changes several parts of the request at once, and a known defect inflates the richer requests.

This paper makes four contributions:

- An investigation-state architecture that gives observations, conjectures and dialogue separate lifetimes, with source provenance, scoped assessments and an explicit obligation to preserve observations across a reset (Section 3).
- A complete, paired comparison of four context policies and two retrieval controls with two LLMs, which yields a negative result for stagnation-triggered resets and an exploratory positive result for an observation-only view (Sections 4 and 5).
- Evidence that ranking accuracy can conceal failed investigations behind a retrieval fallback, together with process measures—reset exposure, final-pool availability, response diagnostics and resource use—that expose such failures (Section 5.4).
- A replication analysis that rebuilds every result table from the saved scores and checks the coverage of all study keys (Appendix B).

The rest of the paper is organized as follows. Section 2 defines the task and introduces a running example. Section 3 presents MORPH-FL. Section 4 describes and justifies the study design, and Section 5 reports its results. Section 6 interprets them, Section 7 discusses threats to validity, Section 8 relates our work to prior research, and Section 9 concludes.

2. Background and Motivating Example

2.1. Report-Based Fault Localization

The input to report-based fault localization is a bug report and a snapshot of the repository before the fix. The output is a ranked list of source locations for a developer to inspect. We consider two granularities: files, identified by repository-relative paths, and functions, identified by qualified names within those files. A ranking says where to look; an explanation also says how an operation could cause the reported failure.

Evidence about an explanation comes from checks of different strength. A source check establishes that a line contains some text, an abstract syntax tree (AST) query establishes that a file contains a syntactic structure, and an execution procedure establishes what the program does for chosen inputs. Throughout the paper, a check *supports* a hypothesis when the observed value equals the value its prediction expects, for example when a predicted expression is present in the inspected code. Support from a source or AST check is therefore a static fact; it does not show that the reported execution reached that code. The investigator sees the report, the pre-fix source and the procedures it is allowed to run. It never sees the reference patch, which the evaluation uses only to derive the target locations.

2.2. Motivating Example

Listing 1 shows a constructed example that we use throughout the paper. A report states that setting `enabled` to false still displays a feature. The handler passes the normalized option to a rendering function, which is omitted. The fault is in `normalize`: the expression `options.get("enabled") or True` evaluates to true when the option is false. Line numbers are counted within each file, so the defaulting return is `config.py:2` and the handler's return is `handler.py:4`.

Listing 1. Constructed configuration fault in two Python files.

```python
# config.py
def normalize(options):
    return {"enabled": options.get("enabled") or True}

# handler.py
from config import normalize
def handle(options):
    config = normalize(options)
    return render(config["enabled"])
```

Because the report describes what is displayed, an investigator may well suspect the handler first. Two explanations then compete. The *bypass* hypothesis claims that the handler renders the raw option instead of the normalized value; it predicts that the handler's return expression mentions `options`. The *defaulting* hypothesis claims that `normalize` replaces an explicit false with a default; it predicts a return expression containing `or True`. Table 1 shows one AST check for each prediction. The first contradicts the bypass prediction and the second supports the defaulting prediction. Neither check executes the program, so the defaulting explanation remains a hypothesis, now with structural support.

Table 1
Two AST checks on Listing 1.

Hypothesis	AST query (expected result)	Result	Assessment
Bypass	A `return` in `handler.py` contains `options` (true)	False	Contradicts
Defaulting	A `return` in `config.py` contains `or True` (true)	True	Supports

Now suppose the investigation stalls and later rounds keep re-proposing the same two checks without learning anything new. Restarting with an empty context would force the investigator to rediscover both results. Keeping only

the leading explanation would be no better if observations were not stored separately: the refutation of the bypass idea could vanish from the next context, and nothing would stop the idea from returning. The example thus captures the dilemma of forgetting in miniature. The stale dialogue should go, but the two observations must stay, and it is unclear whether the competing explanations should stay visible as well. MORPH-FL was designed to make this choice explicit and to keep observations and a portfolio of competing explanations while clearing dialogue; whether the explanations should also be shown in later requests is the empirical question that our study takes up.

2.3. Models at Runtime

To make such retention decisions explicit, MORPH-FL borrows from model-driven engineering (MDE). In MDE, a model represents selected aspects of a system in an explicit structure. A metamodel defines the admissible element types, attributes and relationships, and transformations derive models or views from other models (Schmidt, 2006). Models need not be design-time artifacts: the models@run.time paradigm keeps a model of a running system and uses it to drive the system's adaptation (Blair et al., 2009), much as the monitor–analyze–plan–execute loop of autonomic computing does (Kephart and Chess, 2003). MORPH-FL applies this idea to an investigation rather than a deployed system. Its models describe what the investigator has proposed and observed, and its control loop monitors how they evolve and adapts the investigator's context accordingly.

3. The MORPH-FL Approach

The motivating example calls for three capabilities: a record of checked facts that no reset can erase, a set of competing explanations that can survive a reset without the dialogue that produced them, and a rule for deciding when to reset. This section builds them in that order. After an overview and a shared vocabulary (Section 3.1), it defines the models and their invariants (Section 3.2) and the operations over them (Section 3.3). Sections 3.4–3.7 assemble the operations into the investigation loop, the portfolio, the reset policy and the final ranking, and Section 3.8 replays the running example end to end.

3.1. Overview

Figure 1 gives an overview of MORPH-FL. From a report and a pre-fix repository, MORPH-FL builds a behavior model, a structure model and a fixed pool of candidate files. It then runs a sequence of rounds. In each round it projects the current models into a request for one investigator role, asks the LLM for hypotheses and probes, grounds the proposals in the structure model, runs or reuses the probes and assesses the results. The controller resets the context if the progress monitor detects stagnation, and it stops when the budget or the attempt limit is exhausted. At the end, MORPH-FL ranks files and functions by the scores of the surviving hypotheses.

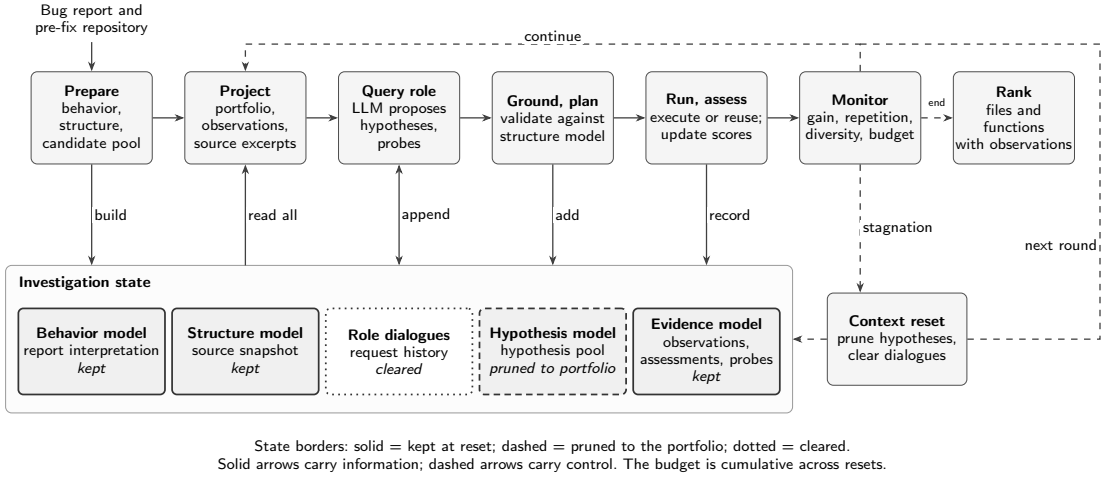


Figure 1: Overview of MORPH-FL. The top row shows one round of the investigation; the bottom panel shows the investigation state and what a context reset does to each part. A reset prunes the hypothesis pool to the portfolio and clears all role dialogues and the progress window; every other part of the state, including budget use, is kept, and the next round projects its request from what remains.

Because our argument turns on the difference between a fact and a conjecture, the following terms are used in exactly one sense. A *hypothesis* is a candidate location together with a proposed fault mechanism and at least one *prediction*, a condition that can be checked. A *probe* is a planned check of a prediction, an *observation* is the recorded result of running a probe, and an *assessment* states whether an observation supports, contradicts or is inconclusive for a hypothesis. Probes, observations and assessments together form the *evidence*. An *investigator role* is an LLM instruction persona with its own *dialogue*, the history of its requests and responses. A *context reset* clears all dialogues and prunes the hypotheses to a *portfolio*. Each iteration of the investigation is an *attempt*, and an attempt whose LLM response is accepted is a *round*. The *epistemic status* of a model element records whether it was observed, derived by analysis or conjectured. We write "LLM" for the language model and reserve "model" for the models of the investigation.

Why use models at all? An LLM dialogue is one undifferentiated artifact. Every fact in it is entangled with the conjecture that prompted it and the reasoning that interpreted it, so shortening it by truncation or summarization can drop facts or blur their status. MORPH-FL instead keeps facts and conjectures as separate elements of models that conform to a metamodel. Controller operations keep their declared status apart (invariant I1 below), every observation survives a reset (I4), and every request is built from the models. The request after a reset is therefore rebuilt from what was kept, not reconstructed from what was said. Between resets, a role's earlier turns are prepended to its projected request, and that dialogue is exactly what a reset clears.

3.2. The Investigation Metamodel

A reset can keep facts and drop conjectures only if the investigation state says which is which. The metamodel therefore partitions the state by the kind of knowledge it holds: what the report claims, what the code contains, what the investigator conjectures and what checks have established. Each part becomes a model with its own element types and lifetime, so that the retention decision of Section 2.2 can be stated per model rather than per message. Figure 2 shows the metamodel.

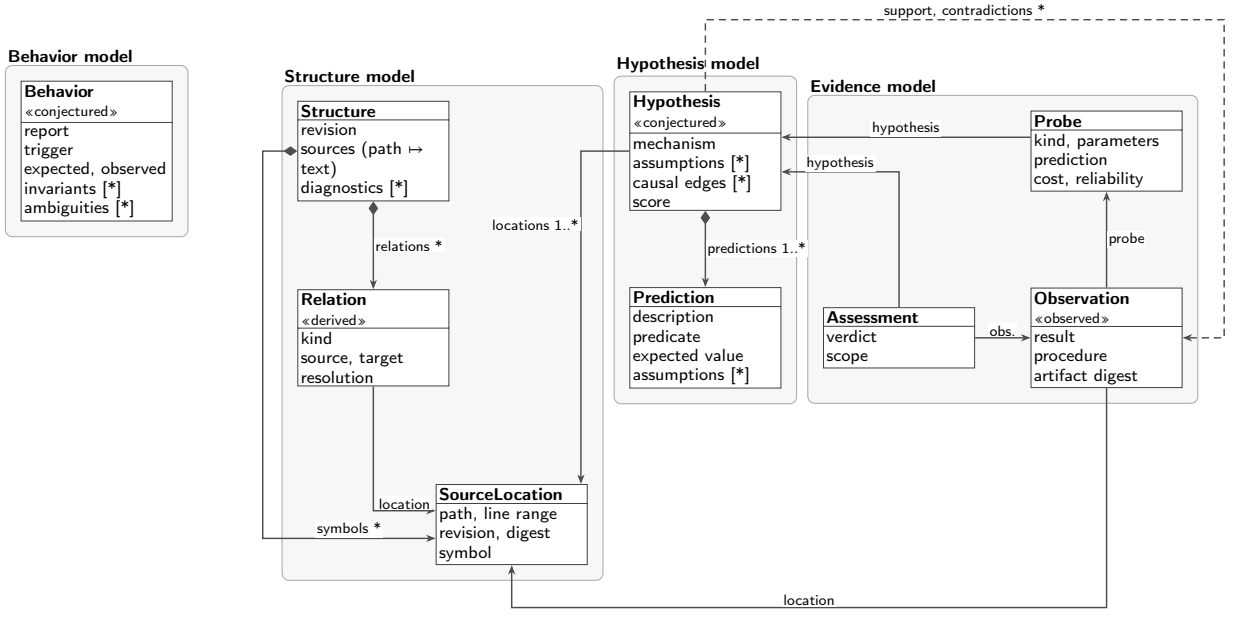


Figure 2: The investigation metamodel. A stereotype abbreviates a status attribute set at creation and never changed: LLM-derived elements are conjectured, extracted relations are derived, and probe results are observed. Diamonds denote containment. The associations from Probe, Assessment and Observation to hypotheses, observations and probes, and the dashed support and contradiction links, are references by identifier; the others hold the referenced objects. Role dialogues are not part of the metamodel.

Behavior model. This model holds the report together with the latest LLM interpretation of it: a trigger, the expected and the observed behavior, candidate invariants and ambiguities. Each accepted LLM response replaces the trigger and the expected and observed behavior. In the evaluated version, ambiguities were appended rather than replaced, a

defect whose consequences we discuss in Sections 4.4 and 7. When the expected or observed behavior is missing, the model records the gap as an ambiguity. The fields are conjectures, but they give every role the same statement of the discrepancy to explain.

Structure model. This model is a snapshot of the production Python files at the fixed revision. It stores each file's text, class and function symbols with their line ranges, and relations: imports, calls, reads, writes and intraprocedural data flow. Every symbol and relation location carries the file's content digest. A call to a top-level definition or an explicitly imported name is marked as syntactically resolved; other calls are marked unresolved. The structure model is built once per run, and it serves to expand the candidate pool and to ground the locations that investigators propose.

Hypothesis model. This model is the pool of live hypotheses. A hypothesis names one file and one or more locations in it, a mechanism in natural language, one or more predictions, optional causal edges (conjectured links along which the fault propagates between locations), links to supporting and contradicting observations, and a score. A prediction consists of a description, a predicate, an expected value and assumptions. A hypothesis is identified by its file, its case-folded mechanism and the kind of probe that accompanies it; when an investigator proposes the same hypothesis again, new locations and predictions are merged into the existing element.

Evidence model. This model holds probes, observations and assessments. A probe names its hypothesis, the prediction it checks, a probe kind and parameters. An observation records the probe's result, the procedure that produced it, the source location with revision and file digest, and the digest of a stored artifact. An assessment links an observation to a hypothesis with a verdict and the scope of what was observed: source presence, static structure or execution. Observations are identified by their content. Controller operations do not modify or delete them, and probes that perform the same check reuse one observation. Probe objects themselves are transient; what persists is each observation with the identifier of its probe, a cache of executed checks and the set of probes already handled.

Table 2 summarizes the four models and what a reset does to each. The dialogues of the investigator roles appear as a fifth row because they are the part of the state that a reset clears, but they are deliberately kept outside the metamodel.

Table 2
The four models of an investigation and their lifetime at a context reset.

Model	Content	Used for	Status	At a reset
Behavior	Report; trigger; expected and observed behavior; invariants; ambiguities	Shared statement of the discrepancy in every request	Conjectured (except the report)	Kept (latest interpretation)
Structure	File texts; symbols; import, call and data-flow relations; digests	Candidate expansion; grounding of proposed locations	Derived	Kept (built once)
Hypothesis	Hypotheses with locations, mechanism, predictions, links to observations, score	Portfolio shown in each request; final ranking	Conjectured	Pruned to the portfolio
Evidence	Observations with provenance; assessments; probe cache and handled probes	Assessment, progress monitoring, projection	Observed	Kept, append-only (probes are transient)
Role dialogues	Requests and responses of each role (not a model)	Conversational continuity within a role	–	Cleared

Separating the models is not enough on its own. An operation that turned a conjecture into an observation, lost the revision of a checked file or dropped an observation at a reset would silently undo the separation. We therefore state four invariants that every operation of Section 3.3 must maintain. I1 keeps facts and conjectures apart, I2 ties every fact to the inspected source, I3 keeps the links between them consistent, and I4 is the preservation obligation of a reset.

I1 *Epistemic separation.* The status of an element is set by the operation that creates it, and no operation changes it. Observations are created only by running probes; relations only by structure extraction; behavior fields only from the report and LLM proposals; hypotheses and causal edges only from LLM proposals. Grounding marks every proposed causal edge as conjectured, whatever certainty the LLM claims, and assessment changes a hypothesis's score and links but not its status.

I2 *Source traceability.* Every hypothesis location and every observation carries the revision and the content digest of its file. The evidence model rejects an observation from another revision, one without procedure and digest provenance, or one whose artifact fails an integrity check, and the controller rejects an observation whose file digest differs from the structure model's.

I3 *Referential integrity.* Every support or contradiction link of a live hypothesis resolves to a recorded observation. The links are checked against the evidence model after each assessment, and against the observations the role may access whenever the hypothesis is projected into a request.

I4 *Observation preservation.* Controller operations neither modify nor delete observations. Insertion checks reject changed records under an existing identifier except for the originating probe identifier; nested result dictionaries are not deeply immutable (Appendix A.2). A context reset leaves the set of observation identifiers unchanged, which the controller asserts at runtime, and it neither refunds nor resets the budget.

The metamodel is implemented as Python dataclasses rather than in a modeling framework such as EMF. I1 holds by construction, because no operation assigns a status after creation and grounding overwrites the status of every proposed causal edge; I2–I4 are enforced by explicit checks in the evidence model, the operations and the controller (Appendix A.1). These checks guard the invariants at runtime, but they are not a proof. The I4 assertion, for example, compares the observation identifiers before and after a reset. Because the reset code never touches the evidence model, the assertion guards against future regressions rather than proving a property, and it is removed when Python runs with optimizations (`-O`). Status is fixed per element type and keeps facts and conjectures apart in requests and assessments; it does not drive control decisions. Assessments of pruned hypotheses remain in the evidence model as history but are no longer shown.

3.3. Model Operations

The models change only through a fixed set of operations, and this is what makes the invariants checkable. Each operation has a declared input and output model, so it is clear which operation may create an observation, which may only create a conjecture and which may only select. In MDE terms, extraction operations are text-to-model and code-to-model transformations, grounding and planning are model-to-model transformations, and projection is a model-to-view transformation whose result is the LLM request. Table 3 lists the operations. In its signatures, r is the report, R and v the repository and revision, z an accepted LLM response with proposals z_j, and ψ_j the probe parameters of proposal j; s, b, h, p, e and a denote a structure model, behavior model, hypothesis, probe, observation and assessment, and H, S, E, B, A and Ω the hypothesis pool, portfolio, evidence model, budget, role dialogues and progress window.

Extraction. EXTRACTSTRUCTURE parses every production Python file with Python's `ast` module, skipping test directories and test files. Files that fail to parse remain retrievable but contribute only a diagnostic.

Candidate preparation. MORPH-FL first ranks all files against the report with BM25, a term-based retrieval function (Robertson and Zaragoza, 2009). The five highest-ranked files seed EXPAND, which collects the files reachable within two hops in either direction over relations not marked unresolved. Data-flow relations stay within a file, so in practice the neighborhood follows imports and calls. The candidate pool consists of the seeds, then the neighborhood files in BM25 order, then the remaining files in BM25 order, truncated to 20 files, and it is fixed for the run. In a larger version of the running example, the import and call from `handle` to `normalize` would bring `config.py` into the pool even if BM25 ranked it below the seeds. Only pool files are ever shown to the investigator, so the pool bounds which files can receive hypotheses.

Grounding and planning. MORPH-FL considers the first four proposals of a response, each with a file, a mechanism, a prediction and a probe. GROUND accepts a proposal only if its file was shown in the request, the proposed line exists in that file, the probe targets the same file, and the mechanism and prediction are nonempty. It then builds a source location with the revision, the file digest and the narrowest enclosing symbol, and it adds the hypothesis to the pool or merges it with an existing one. PLAN turns the investigator's probe specification into a typed probe of one of five kinds: a source quote, an AST query, a test, a trace or a counterfactual comparison of two procedures. The investigator, not MORPH-FL, chooses the check, and MORPH-FL never synthesizes tests from a mechanism. The probe kind is checked when planning and the kind-specific parameters when the probe runs; a hypothesis whose probe proves malformed stays in the pool with score zero.

Running probes. A source-quote probe checks whether a quoted text occurs on a given line. An AST query checks whether the file contains a node of an allowed type, such as a `return` statement, whose source text contains a given

Table 3
Model operations of MORPH-FL.

Operation	Kind	Signature	Responsibility
Extract behavior	Text to model	$(r, z) \mapsto b$	Copy the LLM's interpretation of the report; record missing expected or observed behavior as an ambiguity
Extract structure	Code to model	$(R, v) \mapsto s$	Parse production files; record symbols, relations and digests; keep syntax errors as diagnostics
Expand	Model query	$(s, \mathit{seeds}) \mapsto \mathit{files}$	Two-hop, bidirectional neighborhood of the seed files over relations not marked unresolved
Ground	Proposal to model	$(z_j, s) \mapsto h$	Validate file, line, mechanism and prediction; attach revision, digest and enclosing symbol; mark causal edges conjectured
Plan	Model to model	$(h, \psi_j) \mapsto p$	Build a typed probe from the investigator-specified kind and parameters
Assess	In-place update	$(h, p, e) \mapsto (h', a)$	Compare the observed value with the prediction; link the observation; change the score
Project	Model to view	$(b, S, E, B) \mapsto x$	Build the request view of hypotheses, observations, unresolved predictions and budget
Reset	Selection	$(H, A, \Omega) \mapsto (S, \emptyset, \emptyset)$	Prune the pool to the portfolio; clear dialogues and the progress window; preserve everything else

string; the whole file is searched. Tests, traces and counterfactuals run only as procedures registered by the experimenter in an isolated container, and an unregistered request is recorded as a blocked observation. Two mechanisms avoid repeated work. First, a check, identified by probe kind, file and parameters, executes at most once per run: if another hypothesis already ran it, the recorded observation is reused and assessed for the new hypothesis, but it does not count as new. Second, if the same hypothesis already ran a probe with the same kind and parameters, the probe is skipped and nothing is assessed.

Assessment. ASSESS reads the observed value that the prediction's predicate refers to—for an AST query, whether any node matched—and compares it with the expected value. Equal values support the prediction, unequal values contradict it, and a missing value, a blocked or timed-out procedure, or a predicate that does not fit the probe kind is inconclusive. A hypothesis's score is the signed sum of the weights of its assessments,

$$\mathrm{score}(h) = \sum_{e \in \mathrm{Sup}(h)} w(h, e) - \sum_{e \in \mathrm{Con}(h)} w(h, e), \tag{1}$$

where $\mathrm{Sup}(h)$ and $\mathrm{Con}(h)$ are the supporting and contradicting observations, and $w(h, e)$ is 0.25 when e was assessed for h against a source-presence prediction, 1 for a structural prediction checked by an AST query, and 2 for a prediction checked by execution. The weights encode the relative strength of the three kinds of check; they are chosen ordering priorities, not probabilities. An additive score is the simplest aggregate in which contradictions offset support. It treats observations as independent, a simplification we revisit in Section 7. Each hypothesis–observation pair contributes to the score at most once.

Projection. PROJECT builds the request view from the behavior model, the current portfolio, the observations accessible to the role and the remaining token budget. Each hypothesis appears with its locations, its mechanism labeled as unverified, its predictions, its conjectured causal edges, and the identifiers of its supporting observations (split into static and execution-based) and of its contradicting observations. Predictions of hypotheses without execution support are listed as unresolved. The view lists hypotheses in portfolio order and shows no scores. The controller then adds numbered excerpts of up to four pool files, their BM25 ranks and up to twelve structural relations from those files. The first request shows the four leading candidates; later requests show the BM25 leader and the file of the first portfolio hypothesis and rotate through the rest of the pool. The role's dialogue, if any, precedes the new request. Because projection runs before every request, a reset needs no separate reconstruction step.

3.4. The Investigation Loop

The operations say how the models change, but not in which order or when to stop. A localization investigation must interleave asking the LLM, checking its proposals and deciding whether to continue, while keeping every LLM call inside a cumulative budget and giving the reset policy a defined point at which to act. Algorithm 1 gives the loop that ties the operations together, and we describe it in line order. Lines 1–4 prepare the structure model, the behavior model, the BM25 ranking L and the candidate pool C, and they initialize the hypothesis pool H, the evidence model E (including the probe cache and the set of handled probes), the role dialogues A and the progress window Ω.

Algorithm 1 The MORPH-FL investigation loop

Require: report r; repository R at revision v; attempt limit N; budget B (tokens, probes, time); portfolio bound m
Ensure: ranked files and functions; evidence model E

1: $s \leftarrow \textsc{ExtractStructure}(R, v)$
2: $b \leftarrow \textsc{ExtractBehavior}(r, \varnothing)$
3: $L \leftarrow \mathrm{BM25}(r, s)$; $C \leftarrow \textsc{Pool}(L, \textsc{Expand}(s, L_{1..5}))$
4: $H, E, A, \Omega \leftarrow \varnothing$; $n \leftarrow 0$
5: **for** $i \leftarrow 1$ **to** N **do**
6: **if** $\textsc{Exhausted}(B)$ **then**
7: **break**
8: $\rho \leftarrow \textsc{Role}(i)$; $S \leftarrow \textsc{Portfolio}(H, m)$
9: $F \leftarrow \textsc{Files}(C, L, S, i)$
10: $x \leftarrow \textsc{Project}(b, S, E, B) \oplus \textsc{Excerpts}(F)$
11: $z \leftarrow \textsc{Ask}(\rho, A_\rho, x)$ ▷ charges tokens to B
12: **if** $z = \perp$ **then**
13: **continue** ▷ rejected response
14: $A_\rho \leftarrow A_\rho \cdot (x, z)$; $b \leftarrow \textsc{ExtractBehavior}(r, z)$
15: $n \leftarrow n + 1$ ▷ accepted rounds
16: $(H, P) \leftarrow \textsc{GroundAndPlan}(z, s, H, F)$
17: $(H, E) \leftarrow \textsc{RunAndAssess}(P, H, E, B)$
18: $\Omega \leftarrow \textsc{Record}(\Omega, H, E, P, B)$
19: $H^- \leftarrow H$ ▷ pool before any reset
20: **if** $\textsc{Stagnated}(\Omega, n, B)$ **then**
21: $H \leftarrow \textsc{Portfolio}(H, m)$; $A \leftarrow \varnothing$; $\Omega \leftarrow \varnothing$ ▷ reset
22: **if** $\textsc{ExecutionStop}(H^-, E)$ **then**
23: **break**
24: **return** $\textsc{Rank}(L, H)$, E

Each iteration of the loop is one attempt. The loop ends early when the token, probe or time budget is exhausted (line 6); the LLM backend can also end the run when a request no longer fits the remaining budget or the context window. Roles follow a fixed schedule—behavioral, skeptical, structural, dynamic—and share the models but not their dialogues. The behavioral role separates report claims from observations, the skeptical role tries to falsify the leading hypothesis and keep a distinct alternative, the structural role traces dependencies, and the dynamic role proposes registered tests or traces and falls back to static queries.

Lines 8–10 select the portfolio S and the files F to show and project the request; $\oplus$ appends the source excerpts to the projected view. A response that is not a well-formed object with a list of hypotheses is rejected (line 12): it consumes the attempt and its tokens but changes no state. An accepted response is appended to the role's dialogue A_ρ, refreshes the behavior model and yields the planned probes P (lines 14–16); invalid proposals are skipped individually. Line 17 runs the probes not yet handled, in the order described in Section 3.5, and assesses their results. Line 18 records the round's progress, and lines 20–21 reset the context when the investigation stagnates (Section 3.6). If execution procedures are registered, the run can also stop once the leading hypothesis has support from two distinct procedures, stated assumptions and no contradiction (line 22). This check inspects the pool H^- as it was before any reset in the same round, and in a source-only configuration it never holds. Finally, line 24 produces the ranking (Section 3.7).

3.5. The Hypothesis Portfolio

The portfolio decides which hypotheses the investigator sees. Showing every hypothesis would let near-duplicates crowd out alternatives, and showing only the leader would reproduce the restart-from-best problem of the motivating

example. MORPH-FL therefore selects hypotheses by score and diversity, in the spirit of maximal marginal relevance in retrieval (Carbonell and Goldstein, 1998). Diversity rests on a dissimilarity between hypotheses,

$$D(h,h') = 0.6\left(1 - \frac{|\tau(h)\cap\tau(h')|}{\max(1,|\tau(h)\cup\tau(h')|)}\right) + 0.4\,\mathbf{1}[\pi(h)\neq\pi(h')], \tag{2}$$

where $\tau(h)$ is the set of lowercase alphabetic words of at least three letters in the mechanism of h and $\pi(h)$ is its file. D ranges over $[0,1]$ and combines how differently two mechanisms are worded with whether they concern different files. The portfolio $S = \textsc{Portfolio}(H,m)$ is built greedily. Starting from $S_0 = \varnothing$,

$$\begin{aligned} S_{j+1} &= S_j \cup \{h_{j+1}\},\\ h_{j+1} &= \arg\max_{h\in H\setminus S_j}\Big[\text{score}(h) + \min_{h'\in S_j} D(h,h')\Big], \end{aligned} \tag{3}$$

with the minimum over an empty set taken as 1, until $|S| = m$ or H is exhausted; ties go to the higher score and then the smaller identifier. The first choice is thus the highest-scored hypothesis, and each later choice trades score against difference from those already chosen. Because D is at most 1, diversity reorders only hypotheses whose scores differ by less than one, the weight of a single AST check, and among those it prefers hypotheses that differ in wording or file from the ones already chosen. The default bound is $m = 3$.

Within a round, MORPH-FL runs every unhandled probe. It orders them by a utility that prefers inexpensive static probes, probes whose prediction contrasts with those of other hypotheses, and probes of hypotheses that differ from the rest of the pool (Appendix A.5). The order matters only when the probe or time ceiling is reached in the middle of a round.

3.6. Stagnation Detection and Context Reset

The reset policy is meant to act when observable progress stalls, and its timing involves a trade-off. Resetting too early discards each role's line of reasoning while it is still producing new checks. Resetting too late lets the investigation keep re-proposing the same checks and converging on near-identical explanations, as in rounds 2–4 of the running example. The reset rule therefore watches for the observable signs of the second situation: rounds that add no new observation while the investigation either repeats executed checks or loses diversity among its hypotheses.

After each accepted round t, the monitor records four quantities: the number g_t of new observations that are not blocked; the fraction u_t of the round's probes whose check (kind, file and parameters) had already been executed (zero if the round has no grounded probe); the pool diversity d_t, the mean of D over all pairs of live hypotheses (zero for fewer than two); and the tokens κ_t the round consumed. The window Ω holds the samples of the last W accepted rounds since the start or the most recent reset, and n counts accepted rounds since the start. The investigation has stagnated when

$$\begin{aligned} & n \geq n_0 \ \wedge\ |\Omega| = W && \text{(ready)}\\ \wedge\ & \bar{g} \leq \theta_g \ \wedge\ (\bar{u} \geq \theta_u \vee \bar{d} < \theta_d) && \text{(stalled)}\\ \wedge\ & B_{\text{rem}} > \max\big(\kappa_0, \lfloor \kappa_t/2 \rfloor\big) + \kappa_0 && \text{(affordable)}, \end{aligned} \tag{4}$$

where $\bar{g}$, $\bar{u}$ and $\bar{d}$ are window means and B_{rem} is the remaining token budget. The defaults are $W = 3$, $n_0 = 4$, $\theta_g = 0$, $\theta_u = \theta_d = 0.5$ and a reserve of $\kappa_0 = 256$ tokens. With $\theta_g = 0$, the gain condition requires that the last three rounds produced no new observation at all. Appendix A.7 lists two further conditions: an optional consumed-fraction gate, which is vacuous under the defaults, and a requirement that the window's rounds consumed at least κ_0 tokens, which realistic LLM rounds satisfy.

Each condition encodes a judgment. Judging gain over a window rather than a single round avoids resetting after one unproductive response. Requiring repetition or low diversity is intended to distinguish an investigation that circles, re-proposing executed checks or converging on similar hypotheses, from one that explores without immediate success. The budget condition requires the remaining tokens to exceed a reserve plus half the tokens of the latest round, a heuristic estimate of the cost of a fresh request that does not guarantee that a full further round fits. The thresholds are design parameters rather than calibrated values, and Section 5.2 reports how often the default policy acted in practice.

A reset (line 21 of Algorithm 1) prunes the hypothesis pool to the portfolio, clears every role dialogue and the progress window, and changes nothing else. The behavior and structure models, all observations and assessments, the probe cache, the set of handled probes, the accepted-round count n and the cumulative budget remain. Because the window is cleared while n is kept, two resets are at least W accepted rounds apart.

The effects of a reset unfold at two times. Immediately, the next request no longer carries any dialogue. Because each dialogue turn stores the full request that produced it, a long dialogue holds several stale copies of earlier state, and the reset removes them. The dialogue is, however, also the only place where excerpts of files no longer displayed persist, and it preserves each role's line of reasoning; a reset trades this continuity for a request free of stale state, and the fixed-reset and continuous-context workflows of our study bracket that trade-off. The visible portfolio does not change at the moment of the reset, since the portfolio of a pruned pool is the pool itself. Pruning acts later: pruned hypotheses can no longer re-enter the portfolio, count toward diversity or influence the final ranking. A reset therefore forgets interpretations as well as dialogue. We made this choice deliberately. At a stagnation point, the hypotheses outside the portfolio are those that score and diversity already rank below m others, and pruning them keeps the diversity signal and the final ranking focused on the explanations the investigation is still pursuing. If a pruned hypothesis is proposed again, it returns with a zero score and without its former links, and its earlier probe is not rerun because it has already been handled. Observations remain recorded (I4) and available to the next projection, although I4 does not guarantee that every record fits the backend's context window.

3.7. Producing the Ranking

When the loop ends, RANK orders files by the highest score of a live hypothesis in each file. A file without a live hypothesis receives score -1, and BM25 order breaks ties, so lexical retrieval serves as the fallback. This rule has a consequence that is central to interpreting our results. A file whose hypothesis was proposed but not yet checked (score 0), or contradicted only by a source-presence check (-0.25), still ranks above every uninvestigated file, so the LLM's proposals can reorder the BM25 list without supporting evidence. Conversely, an investigation that ends with no live hypothesis returns exactly the BM25 order. The function ranking lists the enclosing symbols of live hypotheses' locations in decreasing order of score and has no lexical fallback.

The output also contains the observations and assessments, so hypothesis-backed locations can be traced to the checks that support them; retrieval-only entries and unchecked hypotheses have no such support. MORPH-FL reports no confidence value. Its abstention flag is cleared by any supporting execution-based assessment in the run, including one for a non-leading or pruned hypothesis, so the flag does not certify the leading location (Appendix A.8). It remains set for source-only runs.

3.8. The Running Example, End to End

Table 4 follows the running example through Algorithm 1, and Table 14 in Appendix A.7 spells out the reset arithmetic. Scripts in the replication package check both against the implementation. The last row shows what the portfolio adds: with a leader-only bound, the refuted bypass idea could return without the link that records its refutation.

3.9. Scope and Assumptions

Several boundaries apply throughout. MORPH-FL targets Python, and its structure model is approximate. Calls are resolved syntactically, dynamic dispatch stays unresolved, and the two-hop neighborhood is a heuristic rather than a semantic slice; because import targets outside the repository are traversed, widely imported standard-library modules act as hubs at the second hop and can make the neighborhood broad. The stagnation monitor counts an observation as new by its identity, which includes the probe parameters, so a slightly reworded quote counts as new evidence. Scores and dissimilarities are heuristics, not probabilities. The fixed candidate pool bounds recall, and retaining every observation does not bound the size of later requests. Finally, a reset preserves observations but not the interpretations attached to pruned hypotheses. Appendix A documents the remaining operational details.

4. Study Design

The study asks how the retention policy of a localization agent affects its rankings, its use of resources and the investigation it actually carries out. It does not ask whether MORPH-FL outperforms other localizers. This goal, together with the way MORPH-FL produces its output, shaped three decisions that run through the design. We explain them first and then describe the workflows, subjects, runtime and measures.

Table 4
The running example traced through MORPH-FL, with hypotheses h_b (bypass) and h_d (defaulting).

Step	What happens
Prepare	The structure model records `handle` and `normalize`, an import of `config.py`, and a call from `handle` to `normalize`; the call to `render` is unresolved. Both files are BM25 seeds, and `config.py` is also reachable from `handler.py` through the import and call.
Round 1	The behavioral role sees the report and both files. It records the trigger and the expected and observed behavior, and proposes h_b and h_d with AST probes whose predictions use the predicate `ast_matches` with expected value true.
Ground	h_b is placed at `handler.py:4` in `handle`, h_d at `config.py:2` in `normalize`, each with revision and file digest.
Assess	No `return` in `handler.py` contains `options`: h_b is contradicted, score -1. The `return` in `config.py` contains `or True`: h_d is supported, score $+1$.
Rounds 2–4	The skeptical, structural and dynamic roles re-propose the executed checks. Each round adds no observation ($g_t = 0$) and repeats every probe ($u_t = 1$).
Reset	After round 4, $n = 4$, the window is full, $\bar{g} = 0$, $\bar{u} = 1$, and the budget suffices. The pool is pruned to the portfolio (h_d, h_b); dialogues and window are cleared; observations, scores and budget are unchanged.
Round 5	The request carries the behavior model, h_d and h_b, both observations and the remaining budget, but no earlier dialogue.
Rank	`config.py` (score 1) ranks first; `handler.py` ties at -1 with uninvestigated files and is placed by BM25. Functions: `normalize`, then `handle`.
Leader only	With $m = 1$, every request shows only h_d, and the reset discards h_b. If a later round proposes the bypass idea again, h_b returns with score 0 and no links, and its probe is skipped as already handled; the refuting observation stays in the request but is no longer linked to it.

4.1. Design Rationale

Comparing policies within one implementation. A comparison of MORPH-FL against external agents would confound the context policy with differences in tools, prompts, navigation and candidate retrieval, any of which can affect localization accuracy as much as the policy itself. Because MORPH-FL expresses every workflow as a configuration of the same controller rather than as separate code (Appendix A.1), the workflows share grounding, probe execution, scoring, excerpt construction, role instructions and ranking, and they differ only in what requests show and what resets do. The comparison is therefore a controlled contrast of context policies. The price is that it says nothing about competitiveness with complete localization systems, a question we leave to future work.

Pairing on issues. Localization difficulty can be expected to vary far more between issues than between context policies, so we run every workflow on every issue with each LLM and compare workflows issue by issue. Pairing removes issue difficulty from each contrast and lets us report exactly which issues a workflow gains or loses relative to another, instead of only comparing aggregate rates.

Measuring the investigation, not only the ranking. As Section 3.7 explains, an investigation that ends without a live hypothesis returns the BM25 order. A workflow whose investigations fail could thus score exactly like retrieval, and two workflows with equal accuracy could differ completely in how much investigative work stands behind their rankings. We therefore add two retrieval controls that show what the LLM adds to the ranking, and process measures that show whether the investigation produced usable output at all. For the same reason, an adaptive policy can only matter where it acts, so a null result for adaptive resets is uninterpretable without knowing how often resets occurred.

These decisions translate directly into the research questions of Section 1. RQ1 compares the rankings a developer would receive. RQ2 measures reset exposure, which is the precondition for any effect of a reset policy, and uses the fixed schedule to separate the question of *whether* to reset from *when*. RQ3 asks whether a policy trades accuracy for cost under a common ceiling, since policies change request length and output volume. RQ4 examines final hypothesis pools and response diagnostics, which the retrieval fallback makes necessary.

4.2. Context Workflows

Table 5 defines the evaluated configurations. They were chosen to cover the retention space between never forgetting and forgetting everything but the facts. *Continuous context* is the no-forgetting baseline: it keeps every dialogue and every live hypothesis and never resets. *Full adaptive*, the MORPH-FL design of Section 3, projects the model state together with each role's dialogue and resets and prunes on stagnation. *Fixed reset* applies the same reset operation after every third attempt, provided that attempt was accepted, without consulting the monitor; comparing it with full

adaptive separates the effect of resetting from the effect of the monitor's timing. The *observation-only view* sits at the opposite end from continuous context: each request is rebuilt from scratch and shows the report and the recorded observations, but none of the investigator's earlier hypotheses or dialogue. It tests whether showing hypotheses to the investigator helps at all. Finally, the BM25 and graph retrieval controls rank files without an LLM, which measures how much of each workflow's accuracy the retrieval fallback already provides.

Table 5
Evaluated configurations. Dialogue and hypothesis-pool retention are part of the treatment.

Workflow	Artifact name	Memory and reset policy
Full adaptive	`morph`	Project model state plus role dialogue; reset and prune on stagnation
Continuous	`no_forgetting`	Keep dialogue and all live hypotheses; no resets
Fixed reset	`fixed_reset`	Same reset and pruning operation every third attempt if that attempt is accepted
Observation-only	`stateless_summary`	Report and compact observations as memory; no dialogue, resets or pruning
BM25 / Graph	`bm25`, `graph`	File retrieval; no LLM and no function ranking

All four LLM workflows share grounding, source and probe validation, evidence processing, scoring, the rules for constructing source excerpts, role instructions and ranking, and within each LLM they use the same checkpoint and resource ceilings. Because evolving hypotheses influence which files are displayed, the source a workflow shows downstream need not be identical across workflows; this is part of the treatment.

The observation-only view needs a closer description, because it differs from the other workflows in more than one respect. It retains the common numbered source excerpts, retrieval ranks, revision and instructions. It omits hypotheses, the interpreted behavior, unresolved predictions, relations, the displayed budget and detailed provenance. Its artifact name is historical and does not refer to an LLM-written summary; observations are listed as recorded. Internally, the workflow still keeps hypotheses, grounds and checks proposals and uses their scores to rank locations, and because it never resets, its unpruned pool can affect the final ranking. The contrast with the other workflows therefore bundles several changes, and it cannot isolate the effect of removing dialogue, removing hypotheses or omitting any individual field. In addition, all workflows store evidence externally, so none of them measures the benefit of preserving observations against deleting them. We return to these limits in Section 7.

4.3. Subjects

We draw subjects from SWE-bench Lite and SWE-bench Verified (Jimenez et al., 2024; Chowdhury et al., 2024). Both provide real issues from popular Python projects together with the pre-fix repository snapshot and the reference patch, which is what report-based localization requires, and both are widely used to evaluate LLM agents. The two splits contain 300 and 500 issues and share 93, giving 707 unique issues; an issue in both enters our sample once.

We selected 50 issues. A few repositories, Django above all, contribute a large share of the benchmark issues, so a random sample would mostly measure behavior on those projects. We therefore selected issues round-robin across twelve repositories, in deterministic hash order within each repository, after excluding thirteen issues that had been inspected during development. The selected identifiers were fixed before any prediction or label for this selection was opened. Balancing limits the influence of any single project at the price of changing the benchmark's mixture, so the sample is not representative of either split. Its size reflects the cost of a complete paired design: each issue is investigated eight times, and we preferred covering two LLMs on every issue to covering more issues with one. Table 6 gives the repository composition. All 50 issues have file targets and 48 have function targets.

Investigators receive the public report and the pre-fix source. The reference patches are visible only to the evaluator, which derives the changed files and functions as localization targets. These targets are proxies for the fault location rather than adjudicated root causes, as we discuss in Section 7.

4.4. LLMs, Budget and Evidence

LLMs. We use two open code LLMs from different model families, Qwen2.5-Coder-7B-Instruct (Hui et al., 2024) and DeepSeek-Coder-6.7B-Instruct (Guo et al., 2024). Open checkpoints can be pinned to exact commits and run locally, which keeps the study reproducible and free of the silent updates of hosted APIs, and local inference gives

Table 6
Repository composition of the fixed 50-issue sample. Function eligibility is determined separately by nonempty patch-derived targets.

Repository	Issues	Function-eligible
astropy/astropy	5	5
django/django	5	5
matplotlib/matplotlib	5	5
mwaskom/seaborn	4	4
pallets/flask	3	3
psf/requests	4	4
pydata/xarray	4	4
pylint-dev/pylint	4	3
pytest-dev/pytest	4	4
scikit-learn/scikit-learn	4	4
sphinx-doc/sphinx	4	4
sympy/sympy	4	3
Total	50	48

exact token accounting under our own backend. Models of this size are also the setting in which context management should matter most: their effective context is short and their tolerance for long, noisy inputs is limited, so if forgetting helps anywhere, it should help here. Using two families lets us see whether a result depends on the model. Each model runs on one NVIDIA A30 GPU (24 GB) in BF16; the four roles take turns and are not four concurrent models.

Budget. Each investigation has a cumulative ceiling of 32,000 input-plus-output tokens and at most 16 attempts. The per-request limit of 16,384 tokens matches the 16K context window of the DeepSeek checkpoint, so both models face the same request limit, and generation is capped at 1,536 output tokens per request. The ceilings were chosen so that context pressure is real and the reset policy has room to act. The token ceiling is binding in practice, since mean consumption lies between 27,500 and 30,000 tokens for every workflow (Section 5.3), and 16 attempts leave room for several resets after the monitor's earliest opportunity at round $n_0 = 4$. Invalid generations consume their actual tokens. Sampling uses temperature 0.2, top-p 0.95 and top-k 50, which keeps the structured output close to deterministic. The study settings override some library defaults, which Appendix A.9 lists in Table 15.

Evidence. All evaluated probes are source or AST checks. Executing tests would require a reproduced environment for every issue, and the SWE-bench tests that expose each failure were written together with the fix, so registering them would leak the target. Static checks, in contrast, are deterministic, inexpensive and available for every issue, which gives all workflows the same evidence channel. The consequence is that every assessment in the study establishes a static fact rather than an execution-based causal validation, and the execution-based stop of Algorithm 1 never fires.

Runtime and provenance. The evaluated runtime is the frozen v12 release of 20 October 2026, and each investigation uses seed 1. We spent our budget on issues and models rather than on repeated sampling, which means that the study cannot separate workflow effects from sampling variation within an issue; we report exact discordances and descriptive intervals for this reason. Of the 400 predictions, 132 Qwen records were imported from an earlier collection under the same frozen protocol after identity validation, and 268 were newly collected; no record was regenerated or counted twice. A manifest records the selection and protocol digests, and Appendix B records the model and dataset pins. Archived finalization verified 400 investigation and 100 control records on 25 October 2026. Later hardening of the checker added protocol, revision and shard-identity checks, and an inspection of the saved configurations found them consistent with those identities, although the raw contents were not revalidated in full under the hardened checker.

A known defect. The frozen behavior transformation has a prompt-growth defect: repeated unresolved ambiguities can be appended to the behavior model and displayed again. The richer requests show these duplicates, whereas the observation-only view omits the field. We have not quantified how often the defect occurs or how it affects accuracy. A regression-tested correction exists, but it is outside the evaluated runtime and contributes no inference results to this paper.

4.5. Measures and Analysis

Ranking measures. For an eligible issue i, let r_i be the rank of the first patch target in a deduplicated ranking, with $r_i = \infty$ if no target appears:

$$\text{Top-}k = \frac{1}{M}\sum_{i=1}^{M} \mathbb{1}[r_i \le k], \qquad \text{MRR} = \frac{1}{M}\sum_{i=1}^{M} 1/r_i, \quad 1/\infty = 0. \tag{5}$$

File measures use $M = 50$ and function measures $M = 48$. Class-only entries do not count as exact function hits, and empty outputs count as misses rather than being removed. File and function rankings are evaluated independently.

Process and resource measures. For RQ2 and RQ4 we count the runs that end with a nonempty hypothesis pool, the runs with at least one reset and the total and mean number of resets, and, for DeepSeek, the accepted and invalid responses and hypothesis events recorded in the audit logs. For RQ3 we report actual tokens and elapsed seconds. Resource figures cover the calls within each completed investigation; they exclude interrupted or retried attempts, setup, energy and developer effort. Because the two LLMs use different tokenizers, token counts are comparable within a model but not across models.

Analysis. The per-issue outcome of interest is binary, whether a patch file appears first, so we report paired file Top-1 effects as full adaptive minus comparator in percentage points (pp), together with the number of issues that each side alone localizes. Issues from the same repository share code, conventions and reporting style and are therefore not independent. We quantify uncertainty with 95% repository-cluster bootstrap intervals that resample the twelve repositories with replacement while keeping all their issues (Efron and Tibshirani, 1993; Field and Welsh, 2007), using 2,000 draws and Python random seed 20261024. The intervals are unadjusted and descriptive. We do not perform significance tests: the preference for the observation-only view emerged after we had seen the outcomes, and testing a hypothesis formed by inspecting the same data would overstate the evidence. The reset-exposure and paired pool analyses were likewise added during revision and are descriptive. All tables are rebuilt from the saved scores by the analysis script described in Appendix B.

5. Results

We report the results by research question. Throughout, “Qwen” and “DeepSeek” denote the two LLMs and all counts refer to the fixed sample of 50 issues.

5.1. RQ1: Ranking Outcomes

Table 7 summarizes the rankings. With Qwen, the full adaptive, continuous and fixed-reset workflows each place a patch file first on 21 of 50 issues (42%). The observation-only view places one first on 25 issues (50%), and its advantage is not confined to the first rank: file MRR rises from 0.561 to 0.645, Top-10 from 86% to 90%, and function Top-1 from 8 to 11 of 48 issues.

Table 7
Ranking outcomes on all 50 study issues per workflow and LLM (seed 1). Top-5 and Top-10 are percentages; Top-1 cells show exact hits and denominators. Retrieval controls run once per issue and produce no function ranking.

LLM	Workflow	File Top-1	Top-5	Top-10	File MRR	Fn. Top-1	Fn. MRR
Qwen	Full adaptive	21/50	72	86	0.561	8/48	0.177
Qwen	Continuous	21/50	74	86	0.572	8/48	0.177
Qwen	Fixed reset	21/50	76	88	0.577	7/48	0.170
Qwen	Observation-only	25/50	78	90	0.645	11/48	0.271
DeepSeek	Full adaptive	20/50	70	86	0.547	0/48	0.000
DeepSeek	Continuous	20/50	70	86	0.547	0/48	0.000
DeepSeek	Fixed reset	20/50	70	86	0.547	0/48	0.000
DeepSeek	Observation-only	20/50	72	88	0.550	6/48	0.135
BM25 (shared)		20/50	70	86	0.547	–	–
Graph (shared)		20/50	70	86	0.547	–	–

DeepSeek tells a different story. All four workflows reach 20 of 50 issues (40%) at file Top-1, and the full adaptive, continuous and fixed-reset workflows match both retrieval controls on every file measure. The observation-only view is marginally better at Top-5 and Top-10 (72% and 88% against 70% and 86%), and it is the only DeepSeek workflow with any function-level hit, ranking the patch function first on six issues where the others rank it first on none. Equal aggregate scores do not by themselves imply identical rankings, but, as RQ4 will show, the agreement with retrieval here has a simple explanation.

The paired comparison in Table 8 sharpens these observations. Under each LLM, the full adaptive, continuous and fixed-reset workflows hit exactly the same issues at rank one: the per-issue hit indicators agree, not merely their means. Their full rankings still differ, since Qwen's Top-5 and MRR vary across the three workflows. Against the observation-only view, the full workflow gains one issue and loses five with Qwen, a net difference of four issues, or -8 pp; with DeepSeek it gains four and loses four.

Table 8
Paired file Top-1 contrasts: full adaptive minus comparator. Counts concern whether a target is ranked first, not the identity of the top-ranked path. Intervals are descriptive repository-cluster intervals from the saved 2,000-draw analysis.

LLM	Comparator	Full only	Comparator only	Difference (pp)	95% interval
Qwen	Continuous	0	0	0	[0, 0]
Qwen	Fixed reset	0	0	0	[0, 0]
Qwen	Observation-only	1	5	-8	[-16, 0]
DeepSeek	Continuous	0	0	0	[0, 0]
DeepSeek	Fixed reset	0	0	0	[0, 0]
DeepSeek	Observation-only	4	4	0	[-9.8, 12]

The repository-cluster interval for the full-minus-observation-only difference is $[-16, 0]$ pp with Qwen and $[-9.8, +12.0]$ pp with DeepSeek. The degenerate $[0, 0]$ intervals against continuous and fixed reset reflect the absence of any discordant issue in this sample; they indicate neither population-level precision nor equivalence. Compared with retrieval, which also localizes 20 issues at rank one, the full workflow adds one net issue with Qwen and none with DeepSeek.

Answer to RQ1. Resetting the context, adaptively or on a schedule, did not change which issues were localized at rank one relative to continuous context. The observation-only view achieved the best observed Qwen rankings at both file and function level, and it was the only DeepSeek workflow to rank any patch function first. With few discordant issues and a single seed, these differences are descriptive and do not establish superiority.

5.2. RQ2: Reset Exposure

A reset policy can only change an investigation in which it acts. Table 9 shows that the adaptive policy acted rarely with Qwen, in 8 of 50 runs with nine resets in total (0.18 per run), but in most DeepSeek runs, 29 of 50 with thirty resets (0.60 per run). The fixed schedule acted far more often: in all 50 Qwen runs with 52 resets and in 45 of 50 DeepSeek runs with 72 resets. A scheduled reset can be skipped when the third attempt is invalid or the budget ends the run first.

The null result of RQ1 therefore cannot be explained by resets simply not occurring. Fixed resets reached nearly every investigation, and adaptive resets reached a majority of the DeepSeek investigations, yet neither policy changed a single rank-one outcome. Nor do the data show that every reset was ineffective, because lower ranks and trajectories may still have changed. And because the monitor triggers on the evolving state of an investigation, the runs in which it acted were not randomly chosen, so comparing runs with and without resets would not estimate the effect of a reset.

What the monitor reacted to also deserves attention. The diversity of a pool with fewer than two hypotheses is zero by definition, so the stalled condition of Eq. (4) can hold for an empty investigation that simply produces no observations. A reset event is therefore not evidence that the investigation escaped a wrong explanation. Likewise, the runtime assertion that observation identifiers survive a reset verifies that records were retained, not that reasoning improved.

Answer to RQ2. Adaptive resets occurred in 16% of Qwen runs and 58% of DeepSeek runs, and fixed resets in 100% and 90%. Despite this exposure, neither reset policy localized any issue at rank one that continuous context missed. For the tested configurations this is a negative result, although it does not show that resetting is unnecessary in general.

Table 9
Reset exposure and final-pool availability. An empty pool returns the retrieval file order. A nonempty pool establishes source-valid proposals, not correct causal explanations.

LLM	Workflow	Runs with reset	Total resets	Mean resets	Final pools
Qwen	Full adaptive	8/50	9	0.18	49/50
Qwen	Continuous	0/50	0	0.00	49/50
Qwen	Fixed reset	50/50	52	1.04	49/50
Qwen	Observation-only	0/50	0	0.00	50/50
DeepSeek	Full adaptive	29/50	30	0.60	6/50
DeepSeek	Continuous	0/50	0	0.00	6/50
DeepSeek	Fixed reset	45/50	72	1.44	7/50
DeepSeek	Observation-only	0/50	0	0.00	47/50

5.3. RQ3: Tokens and Elapsed Time

Table 10 reports the mean resources per investigation. Every workflow consumes most of the 32,000-token ceiling, which suggests that the budget, rather than the attempt limit, usually ends an investigation. Compared with continuous context, the adaptive policy uses slightly more tokens (28,257 against 27,573 with Qwen and 29,197 against 27,912 with DeepSeek) and slightly more time (93.6 against 89.5 seconds and 55.0 against 48.3 seconds). Since it gains no rank-one hit, nothing offsets this small additional cost.

Table 10
Mean resources per completed investigation. Tokens include input and output; elapsed seconds exclude interrupted or retried attempts. Different tokenizers limit cross-model token comparisons.

LLM	Workflow	Total tokens	Output tokens	Seconds
Qwen	Full adaptive	28,257	3,138	93.6
Qwen	Continuous	27,573	3,025	89.5
Qwen	Fixed reset	29,114	3,222	94.7
Qwen	Observation-only	29,325	5,150	140.3
DeepSeek	Full adaptive	29,197	920	55.0
DeepSeek	Continuous	27,912	812	48.3
DeepSeek	Fixed reset	29,553	961	53.8
DeepSeek	Observation-only	29,920	2,506	111.5

The observation-only view is the most expensive workflow in time. It uses 29,325 tokens and 140.3 seconds with Qwen and 29,920 tokens and 111.5 seconds with DeepSeek, roughly 1.5 and 2.0 times the time of the adaptive workflow. The total token counts differ little because all workflows run close to the ceiling, but their composition differs: mean output tokens rise from 3,138 to 5,150 with Qwen and from 920 to 2,506 with DeepSeek. Since generating a token costs far more time than reading one, a larger share of output plausibly accounts for much of the additional time, and it is consistent with the larger number of proposals that RQ4 reports. We did not, however, time individual phases, so this attribution remains an interpretation. Retrieval uses no LLM tokens at all.

Answer to RQ3. Under a common, binding token ceiling, the adaptive policy showed no token or time benefit over continuous context. The observation-only view's better outcomes came with 1.5 to 2 times the elapsed time, driven by much larger output volumes.

5.4. RQ4: Investigation Output Beyond Accuracy

The final pools in Table 9 explain the DeepSeek rankings of RQ1. Under the full adaptive and continuous workflows, DeepSeek ends with a nonempty hypothesis pool on only 6 of 50 issues, and under fixed reset on 7. On every other issue the investigation leaves nothing behind, so MORPH-FL returns the BM25 order, which is why these workflows match retrieval on every file measure and rank no function at all. Under the observation-only view, DeepSeek ends with a nonempty pool on 47 of 50 issues, a difference of 41 issues (82 pp) from the adaptive workflow.

Paired by issue, 43 issues have a pool only under observation-only, two only under adaptive, four under both and one under neither. Qwen, in contrast, ends with a pool on 49 of 50 issues under every richer view and on all 50 under observation-only, so for Qwen the difference between workflows lies in the content of the pools rather than in their existence.

Table 11 traces where the DeepSeek investigations lose their output. The adaptive workflow produced 218 accepted responses but only 22 hypothesis events, whereas the observation-only view produced 293 accepted responses and 259 hypothesis events. Invalid responses were 68 and 50, respectively. Malformed output therefore explains little: most adaptive responses were well-formed, accepted and still contained no proposal that could be grounded. Hypothesis events can repeat earlier proposals and are not correct explanations, and the number of requests need not equal accepted plus invalid responses, because a request can end at a context or token limit. We therefore do not derive a per-response conversion rate from these totals.

Table 11
DeepSeek output diagnostics from archived audit counts. Events may exceed case counts; hypothesis events are not distinct final hypotheses or correct explanations.

Workflow	Accepted responses	Invalid responses	Hypothesis events	Final pools
Full adaptive	218	68	22	6/50
Continuous	193	63	19	6/50
Fixed reset	226	69	25	7/50
Observation-only	293	50	259	47/50

A nonempty pool shows that an investigation produced source-valid proposals; it does not certify that they are correct. The function hits of the observation-only view nonetheless suggest that at least some of its additional DeepSeek proposals point to the right code. Because the two LLMs were run on the same subjects, their results must not be pooled into a single accuracy estimate.

Answer to RQ4. File accuracy can conceal an investigation that produced nothing. With DeepSeek, three workflows matched retrieval because their investigations almost never left a hypothesis behind, while the observation-only view produced grounded hypotheses on nearly every issue and the only function-level hits. Final-pool availability, response diagnostics and function measures distinguish workflows that file Top-1 cannot. The cause of the difference remains confounded by the bundled changes of the observation-only view and by the ambiguity defect.

6. Discussion

6.1. Does Forgetting Help?

The answer depends on what is forgotten. Forgetting dialogue when the investigation stagnates, which is what the MORPH-FL design does, did not help: with both LLMs, adaptive and fixed resets localized exactly the issues that continuous context localized, at slightly higher cost. Forgetting the investigator's own conjectures, which is what the observation-only view does by withholding hypotheses and dialogue from every request, was associated with better Qwen rankings and with far more productive DeepSeek investigations, at a substantially higher time cost. The tension between these two results is the main finding of the study. The question that seemed central when we designed MORPH-FL, when to reset, turned out to matter less than the question of what each request shows.

The RQ2 and RQ4 results together suggest why resets made so little difference. With Qwen, the adaptive policy rarely acted, so it had few opportunities to change anything. With DeepSeek it acted often, but mostly on investigations that had produced no surviving hypothesis, and for those investigations the ranking is the BM25 order regardless of what happens to the context. A reset cannot improve an investigation that has nothing to rank, and clearing the dialogue did not make DeepSeek produce groundable proposals. The observation-only view did, which points to the content of the request rather than its history as the variable that matters.

The architecture of MORPH-FL is what makes this distinction expressible. Observations remain inspectable after a reset in every workflow, so retention of facts was held constant and its benefit is not measured here. What varied was the visibility of conjectures and dialogue. Withholding hypotheses from a request while keeping them internally for grounding and ranking is not the same as pruning them, and the term "forgetting" is only meaningful once it is tied to an explicit boundary of this kind.

6.2. Why Might the Observation-Only View Help?

Several explanations fit the data, and our design cannot separate them. Earlier hypotheses may anchor later proposals, so that an investigator shown its own conjectures elaborates them instead of looking elsewhere. A compact request of observations may simply be easier for a small LLM to process than one that also carries interpreted behavior, relations, predictions and provenance. Removing the dialogue changes how often an accepted response contains no proposal at all. Omitting the defective behavior field avoids the duplicated ambiguities that inflate the richer requests. And the unpruned pool of the observation-only view feeds a different set of hypotheses into the final ranking. Sharing the same JSON output requirements does not remove such input-complexity effects (Tam et al., 2024). The observation-only view is therefore best regarded as a candidate configuration for a corrected, controlled replication rather than as a validated causal treatment. Correcting the ambiguity defect alone will not isolate the other changes; a stronger design would vary the request fields, their serialization, the dialogue and pruning one at a time while controlling inference settings and source exposure.

6.3. Implications

For engineers building localization agents, the output of an agent like MORPH-FL is best understood as an unverified shortlist whose entries can be traced to source checks, and our results suggest that the content of each request deserves at least as much engineering attention as the policy for clearing it. For researchers evaluating such agents, the study yields four practical recommendations.

- Report the share of rankings that come from a retrieval fallback, together with exact function-level scores, next to file accuracy. A good retrieval rank should not be credited as evidence of reasoning.
- Report how many runs a reset policy actually affects, not only the total number of reset events. A mean of 0.6 resets per run hid the difference between low exposure with Qwen and majority exposure with DeepSeek.
- Distinguish accepted but empty responses, invalid responses, grounded proposals and final retention. A response that satisfies the schema need not advance the investigation.
- Compare actual tokens, output volume and elapsed time, not only the budget ceiling. A shared ceiling does not equalize cost, and the most accurate workflow here was also the slowest.

We did not measure developer time saved, repair success or the correctness of the explanations, and the study involved no industrial deployment. Larger LLMs may also process richer views differently. The contribution of the study is reproducible negative and diagnostic evidence for the tested configurations, together with an architecture precise enough to define the next experiments.

6.4. Next Steps

The most immediate step is to quantify the ambiguity duplication and the output funnel on development traces and then to evaluate a separately versioned correction under matched settings. Confirming the observation-only result requires subjects untouched by outcome-informed changes, such as decontaminated benchmarks. Testing whether resets help an investigation recover requires subjects with explicit plateau opportunities and incorrect initial leads, not merely more ordinary runs. Finally, comparing MORPH-FL with a complete upstream localizer and an independent, unstructured tool-using agent would test competitiveness beyond our controlled harness.

7. Threats to Validity

7.1. Construct Validity

The localization targets are the files and functions changed by the reference patch, which are unreviewed proxies for the fault: a fix can touch supporting code or omit an indirectly responsible location. Two issues lack function targets and are excluded from function measures only. Source and AST observations establish static facts rather than reproduced executions or causal diagnoses, and grounding and pool availability measure whether proposals are admissible against the source, not whether they are true. Scores and lexical or path-based diversity are heuristics, not calibrated probabilities, and the additive score treats observations as independent although several observations of one hypothesis can share a cause. An unsupported proposal can outrank an uninvestigated file (Section 3.7). MORPH-FL reports no confidence value, and its abstention flag does not certify the leading location.

7.2. Internal Validity

The strongest contrast in the study, between the observation-only view and the other workflows, changes request fields, provenance verbosity, dialogue, resets and pruning together. The ambiguity defect is demonstrated but unquantified, and it affects only the richer requests. We report the frozen results and disclose this imbalance rather than infer an effect of removing hypotheses. Adaptive reset exposure differs by model, and because the monitor reacts to the state of an investigation, its interventions are not randomized. The shared controller controls engineering differences between workflows, but it also means the study contains no comparison with an independently built agent.

The selection excluded the issues inspected during development, but 132 Qwen predictions were reused from an earlier collection, and the preference for the observation-only view was formed after inspecting outcomes. The study is therefore exploratory rather than an untouched confirmation. Archived identity checks and receipts establish the stated collection provenance, but our local audit does not freshly verify all raw signatures, and the metadata inspection after the checker hardening of 28 October 2026 was not a replacement for content-level verification. The released artifact must include raw sources, configurations, receipts and the permissible traces.

7.3. External Validity

The sample contains 50 issues from twelve Python repositories, two LLMs of about 7B parameters and one seed. Repository balancing changes the benchmark mixture, so neither the full benchmark splits nor an industrial population is represented. Public issue reports may leak the fix or may have appeared in the LLMs' training data (Liang et al., 2026; Aleithan et al., 2024), and pinning inputs does not remove contamination; decontaminated subjects such as SWE-rebench would test a different boundary (Badertdinov et al., 2025). Other programming languages, larger LLMs and execution-based evidence are outside the scope of our claims.

7.4. Conclusion Validity and Reproducibility

Paired counts are exact for the saved outputs, but their generalization is uncertain. The intervals rest on twelve clusters and are unadjusted and descriptive, and zero discordance in this sample does not imply equivalence under other seeds or inputs. A single seed does not estimate stochastic variation, and nondeterministic inference can change token counts even in runs without a reset, so we attribute no difference solely to a reset event. The exposure and pool analyses added during revision are descriptive and do not retroactively change the protocol. Resource means exclude interrupted or retried attempts, setup, energy and developer effort, and the tokenizers of the two LLMs differ. Our audit checks the complete set of score keys, the aggregates and the tables, but it does not reconstruct rankings from raw model outputs. The exact evaluated dependencies and a durable public release remain requirements, and the shallow immutability of dataclasses and the operational retention checks are not formal proofs of semantic preservation.

8. Related Work

We relate MORPH-FL to five lines of work: techniques that rank locations from fixed evidence, LLM-based localization agents, hypothesis-driven debugging, memory and context management for LLM agents, and models in fault localization and at runtime. Table 12 summarizes how the approaches closest to ours manage state, as described in the versions we examined.

8.1. Ranking from Reports, Executions and Learned Representations

Most classical fault-localization techniques rank program elements from evidence that is fixed before ranking begins (Wong et al., 2016; Chang et al., 2025). Spectrum-based formulas such as Tarantula and Ochiai, for example, score statements by their coverage in passing and failing tests (Jones and Harrold, 2005; Abreu et al., 2007). BugLocator combines report–source similarity with the fixes of similar past reports (Zhou et al., 2012), and KBL improves the report used as a query by learning which keywords matter and expanding them with historical ones (Cai et al., 2025). Execution-based techniques rank elements by their behavior in passing and failing runs: DeepRL4FL learns from coverage, statement dependencies and code (Li et al., 2021), and SmartFL reasons probabilistically about the correctness of values along dynamic dependencies (Wu et al., 2025). LLMAO predicts faulty lines from code alone, using adapters on a pretrained code language model (Yang et al., 2024a), and SweRank trains a retrieve-and-rerank model that ranks code for an issue description in one pass (Reddy et al., 2026). MORPH-FL uses lexical retrieval to seed and order its candidate pool and as the fallback order for files without a live hypothesis, but its ranking is the outcome of an investigation whose state evolves. The question we study, what that state should retain across a

Table 12
State and context policies of closely related approaches, as described in the examined versions.

Approach	Retained information	Context policy	Difference from MORPH-FL
AutoFL (Kang et al., 2024)	Current run; repeated predictions	Tool exploration; aggregation across runs	Combines runs; MORPH-FL manages state within one run
SoapFL (Qin et al., 2025)	Analysis results and task outputs	State storage feeds task prompts	Carries task state; MORPH-FL also defines what survives a dialogue reset
Agentless, RGFL (Xia et al., 2025; Sepidband et al., 2026)	Current issue and candidate code	Hierarchical narrowing; candidate reasoning	Narrows by stage; MORPH-FL iterates with assessed checks
LocAgent, ARISE (Chen et al., 2025; Seddik et al., 2026)	Repository structure; retrieved context	Graph tools; structural and data-flow context	Decide what can be inspected; MORPH-FL decides what is retained
MemFL, Repository Memory, AdaRepair-Mem (Yeo et al., 2025; Wang et al., 2026a; Luo et al., 2026)	Project knowledge, history, earlier repairs	Retrieve summaries, history or experiences	Knowledge from other tasks; MORPH-FL retains current-task observations
OrcaLoca (Yu et al., 2025)	Current localization run	Priority-scheduled actions; context pruned by code distance	Prunes by relevance; MORPH-FL separates by epistemic status and resets on stagnation
Ledger (Wang et al., 2026b)	Current repair execution	State view rendered alongside history; repeated actions mediated	Closest state precedent; MORPH-FL clears dialogue and keeps competing hypotheses
Reflexion, MemGPT (Shinn et al., 2023; Packer et al., 2024)	Trial feedback; external memory	Reflections; paging and summaries	Retain prose; MORPH-FL keeps check results as observations
Discovery forgetting (Tan et al., 2026)	Optimization episodes	Restart from the best evaluated candidate	Relies on an objective; MORPH-FL keeps a diverse portfolio
MORPH-FL	Current localization investigation	Projection from models; reset on stagnation	All observations and a hypothesis portfolio under a cumulative budget

reset, does not arise when ranking happens once from fixed evidence. Stronger retrieval or execution evidence would complement MORPH-FL, whose evaluated configuration does not supply the runtime evidence that execution-based methods use.

8.2. LLM-Based Localization Agents

AutoFL lets an LLM explore a repository with tools, explain its prediction and aggregate predictions over repeated runs (Kang et al., 2024). SoapFL, first released as AgentFL, organizes localization as a standard operating procedure of comprehension, navigation and confirmation, passing analysis results between steps through a state store (Qin et al., 2025), and FlexFL narrows candidates in two stages with open-source LLMs and agent tools, drawing on bug reports and tests (Xu et al., 2025a). LLM4FL divides a large code base among several agents and refines their rankings by reflection (Rafi et al., 2024), and Agentless narrows the search hierarchically from files to code elements to edit locations (Xia et al., 2025). AutoCodeRover searches code through program-structure-aware interfaces before patching (Zhang et al., 2024); SWE-agent, OpenHands and CodeAct give agents interfaces or executable actions for repository navigation and editing (Yang et al., 2024b; Wang et al., 2025, 2024); and MASAI splits the work among sub-agents with separate contexts, one of which localizes edits (Arora et al., 2024). RGFL generates candidate-specific reasoning to rank files and elements (Sepidband et al., 2026). LocAgent, CoSIL and ARISE navigate repository graphs,

and ARISE adds definition–use relations and token-budgeted evidence bundles (Chen et al., 2025; Jiang et al., 2025; Seddik et al., 2026). OrcaLoca comes closest to MORPH-FL in managing what the agent sees: it schedules actions by priority and prunes context by code distance (Yu et al., 2025), whereas MORPH-FL partitions context by epistemic status and resets it on stagnation.

Together these systems show that LLM investigations benefit from tools, program structure and intermediate state, so external state alone does not distinguish MORPH-FL. Their contributions concern how an agent navigates, reasons and narrows the search. MORPH-FL addresses a complementary question: which parts of an investigation survive when its dialogue is cleared, and how checked facts stay apart from the conjectures that motivated them. Its structural neighborhood is deliberately simpler than the graphs of LocAgent or ARISE, and a richer navigation backend could feed the same controller. Recent studies of LLM localization in novice programs connect to our results from a different direction. Xu et al. examine the potential and limitations of LLMs in that setting (Xu et al., 2026), and SHARE-RAG combines structure-aware and hypothesis-guided retrieval (Wu et al., 2026). Their task differs from report-based localization in a pre-fix repository, and our observation-only result concerns the lifetime of state within one investigation; it does not imply that hypothesis guidance harms localization in general.

8.3. Hypothesis-Driven Debugging and Diagnosis

Scientific debugging frames debugging as a cycle of hypothesis, prediction, experiment and conclusion (Zeller, 2009). AutoSD prompts an LLM to follow this cycle, using a debugger to test its predictions and producing explanations for developers (Kang et al., 2025), and RepairAgent guides hypothesis-driven repair with a finite-state machine over tool use (Bouzenia et al., 2025); both keep the dialogue history as their state. Model-based diagnosis formalizes the underlying distinction between a system description and observations (Reiter, 1987), and GDE maintains several candidate diagnoses against a shared set of observations while choosing the next measurement by its expected information (de Kleer and Williams, 1987). MORPH-FL inherits the hypothesis–prediction–observation cycle and, like GDE, keeps several candidates against one observation base. It does not offer the guarantees of model-based diagnosis, since its mechanisms are natural language, its structure model is approximate and its probe ordering is a heuristic rather than an information-theoretic choice. What it adds to the cycle is an explicit retention and reset policy, whose benefit for localization our study did not establish.

Keeping several candidate explanations consistent with a shared, growing set of facts also recalls blackboard architectures such as Hearsay-II, in which independent knowledge sources post and revise hypotheses on a shared store (Erman et al., 1980), and assumption-based truth maintenance, which records the assumptions under which each conclusion holds (de Kleer, 1986). MORPH-FL's roles play the part of knowledge sources over a shared evidence model, without the formal dependency tracking of an ATMS. Provenance models such as PROV-DM describe how entities derive from activities (Moreau and Missier, 2013), and MORPH-FL records a comparable trail for each observation—revision, file digest, procedure and artifact digest—so that observations remain usable after the dialogue that requested them is gone.

8.4. Memory and Context Management for LLM Agents

Because LLMs do not use long contexts uniformly (Liu et al., 2024; Hong et al., 2025), agent designers manage what an agent keeps in view. Prompt compression removes tokens that an LLM can do without (Jiang et al., 2023). MemGPT moves information between a bounded context and external storage, evicting and summarizing messages (Packer et al., 2024), and ACON learns natural-language guidelines for compressing an agent's history and observations (Kang et al., 2026). Generative Agents, MemoryBank and A-Mem keep long-term memory stores that agents retrieve from and reflect on (Park et al., 2023; Zhong et al., 2024; Xu et al., 2025b), and Reflexion retains verbal reflections on feedback across trials (Shinn et al., 2023). Ledger keeps observed source, modifications and commands in an execution ledger, renders a current-state view and mediates redundant actions (Wang et al., 2026b). It is the closest precedent for separating mechanically recorded state from reasoning, although its view supplements rather than replaces the interaction history.

SWE-agent makes the opposite choice to our observation-only view: it collapses older observations in the agent's history while keeping its reasoning (Yang et al., 2024b). Lindenbauer et al. find that such observation masking matches LLM summarization on SWE-bench at about half the cost (Lindenbauer et al., 2025), and HiAgent keeps full action–observation pairs only for the current subgoal while summarizing completed ones (Hu et al., 2025). The observation-only view inverts that choice, keeping every observation and dropping the reasoning. Its observed advantage is suggestive, but the bundled contrast and the known defect prevent us from attributing it to the inversion itself.

Memory for debugging and repair often draws on knowledge from outside the current task. MemFL supplies project summaries and debugging guidance refined on pseudo-training bugs (Yeo et al., 2025), Repository Memory retrieves historical commits, linked issues and summaries of active files (Wang et al., 2026a), and AdaRepair-Mem selects earlier repair experiences with coverage-, quality- and stage-aware criteria (Luo et al., 2026). Restarting is a policy of its own: in combinatorial search, randomized restarts counter heavy-tailed runtimes caused by early bad choices (Gomes et al., 1998), and recent work on automated discovery clears agent sessions while keeping the best evaluated candidate (Tan et al., 2026). MORPH-FL differs from these policies in what it retains. It keeps observations from the current task at the target revision rather than knowledge from earlier tasks, it clears the dialogue instead of summarizing it, and it resumes from a portfolio of competing hypotheses rather than from the best candidate, because a localization score, unlike an evaluated objective, does not establish which explanation is correct. Historical memory could still inform hypothesis generation in MORPH-FL, provided that observations from earlier revisions remain distinguishable from checks on the target snapshot.

8.5. Models in Fault Localization and at Runtime

MDE research has localized faults in Simulink models and model transformations (Liu et al., 2016; Troya et al., 2018); in those settings the model is the artifact being debugged, whereas here the models describe an investigation of ordinary Python code. The closer lineage is models@run.time, which keeps explicit models to monitor and adapt a running system (Blair et al., 2009), and the monitor–analyze–plan–execute loop of autonomic computing (Kephart and Chess, 2003). MORPH-FL applies that pattern to investigation state and context. Its lightweight dataclasses, explicit checks and source links provide operational traceability without a dedicated transformation language or formal proof, and the available comparisons do not isolate their value as a representation.

9. Conclusion

We set out to ask whether forgetting helps an LLM agent localize faults, and we built MORPH-FL so that the question could be asked precisely: observations, hypotheses and dialogue live in separate runtime models, a reset can clear one while preserving another, and every request is a view of explicit state. On a complete, paired study of 400 investigations with two LLMs and 100 retrieval controls, the answer was not the one the design anticipated. Resetting the dialogue when the investigation stagnated, or on a fixed schedule, did not localize a single additional issue at rank one compared with continuous context. Showing the investigator only its observations, and neither its hypotheses nor its dialogue, raised Qwen's file Top-1 from 42% to 50% and turned DeepSeek investigations that almost never left a hypothesis behind into ones that nearly always did, although it took considerably longer.

Two lessons follow. For agent design, what a request shows appears to matter more than when its history is cleared. For evaluation, file rankings alone can hide failed investigations behind a retrieval fallback, so reset exposure, grounded output, function hits and resource use should be reported alongside accuracy. The observation-only result is post hoc and confounded by bundled changes and a known defect; confirming it, and isolating which part of the view is responsible, requires a corrected runtime, factor-isolating designs and independent subjects. We hope the architecture and the replication package make those experiments straightforward to run.

A. Implementation Details

This appendix documents the operational details of the prototype that the main text summarizes. Paths are relative to `src/morph_fl/`.

A.1. Code Correspondence

A reader who wants to check a claim about MORPH-FL against its code needs to know where each step of the algorithm lives. The prototype is a Python package, `morph_fl`, and Table 13 maps each line group of Algorithm 1 and each operation of Table 3 to the function that realizes it. All workflows of Table 5 run through the same controller, `LocalizationController.run` in `search/controller.py`; a workflow is a configuration object (`search/config.py`), not separate code. The observation-only view, for example, is the configuration `forgetting="stateless", model_memory=False`: the controller still calls PROJECT but replaces the projected view with the report and the list of accessible observations before sending the request, and it rebuilds each request from scratch instead of appending to a dialogue.

Table 13
Where each step of Algorithm 1 is realized in the implementation. Paths are relative to `src/morph_fl/`; all steps execute inside `LocalizationController.run` (`search/controller.py`).

Algorithm lines	Step	Implementation	Invariant or check
1	ExtractStructure	`analyze_python` (`program_analysis/python.py`)	I2: file digests recorded
2	ExtractBehavior	`complaint_to_contract` (`transformations/pipeline.py`)	I1: fields conjectured
3	BM25, Expand, Pool	`bm25` (`program_analysis/retrieval.py`), `compose_slice` (`pipeline.py`)	Pool fixed for the run
6	Exhausted	token, probe and time checks at the top of the loop	64-token admission floor
8	Role, Portfolio	role schedule in `SearchConfig`; `select_portfolio` (`search/policies.py`)	Eq. (3)
10	Project, Excerpts	`fresh_context` (`pipeline.py`); `source_window` (`controller.py`)	I3: links checked on projection
11–12	Ask	`InvestigatorSessions` (`agents/sessions.py`); BF16 Transformers backend (`experiments/full/inference.py`)	Budget charged per call
16	Ground, Plan	`synthesize`, `hypothesis_to_probe` (`pipeline.py`)	I1, I2: causal edges conjectured
17	RunAndAssess	`rank_probes` (`policies.py`); `ProbeExecutor` (`investigation/executor.py`); `evidence_update` (`pipeline.py`); `EvidenceLedger` (`investigation/ledger.py`)	I2, I4: insertion checks; Eq. (1)
18–20	Record, Stagnated	`PlateauMonitor.update`, `PlateauMonitor.evaluate` (`policies.py`)	Eq. (4)
21	Reset	reset branch of the controller; `select_portfolio`	I4: runtime assertion on observation identifiers
24	Rank	ranking construction at the end of `run`	BM25 fallback for files

Data model and integrity. The metamodel of Fig. 2 is implemented in `domain/models.py` as the dataclasses `BehavioralModel`, `SystemModel`, `Relation`, `SourceLocation`, `FaultHypothesis`, `Prediction`, `Probe`, `Evidence` (an observation) and `Assessment`, with a `Status` enumeration holding the values observed, derived and conjectured. Dataclass annotations are not enforced at runtime, so I2–I4 rest on explicit checks: insertion checks in `EvidenceLedger`, digest comparisons in the controller, and a Python assertion after every reset that the set of observation identifiers is unchanged. Every request and response is stored as a content-addressed artifact, and a hash-linked audit log records each step, so a run can be replayed and inspected after the fact.

Checking the paper against the code. Two scripts in the replication package check the running example of Table 4 and the reset arithmetic of Table 14 against the implementation, so the worked examples in this paper are executable rather than merely illustrative.

A.2. Realization of the Ledger and Reset

Two details complete the mapping of Table 13. The ledger rejects a changed observation under an existing identifier, except for the identifier of the originating probe, and nested result dictionaries are not deeply immutable. The reset itself combines `select_portfolio` with the controller's reset branch, which clears the role sessions and the monitor window.

A.3. Structure Extraction and Requests

Structure extraction skips symbolic links, files whose names start with `test_`, and the directories `tests`, `test`, `.git`, `venv`, `.venv`, `__pycache__` and `node_modules`. A file with a syntax error stays in the source map with a diagnostic but contributes no symbols or relations. Import relations and syntactically resolved calls are marked resolved even when their target lies outside the repository. EXPAND traverses such targets but returns only repository files, so two files that import the same external module become neighbors at the second hop.

Each excerpt holds 40 numbered lines, capped at 4,500 characters. When the report names a function of the file, the excerpt starts at that function's body, after any docstring; otherwise it is centered on a stack-trace line for the file, if the report gives one, and else on the line with the greatest identifier overlap with the report. The relations shown are the first twelve import, call or data-flow relations, in extraction order, whose source file is displayed. A proposal's file must be displayed, but its line need only exist in the file. Requests also carry the revision, the identifiers of registered procedures and a fixed instruction to recheck recorded contradictions and to quote numbered source.

A.4. Grounding, Identity and Reuse

The proposed line defaults to 1. The enclosing symbol is the narrowest class or function whose line range contains the line, so a location inside a class but outside its methods yields a class-only entry. A prediction given as a plain string defaults to the predicate `source_contains` with expected value true. Hypothesis identity hashes the file, the case-folded mechanism and the probe kind, so the same mechanism checked by two probe kinds yields two hypotheses. Logical probe identity hashes the hypothesis identity, the kind and the parameters but not the prediction. The physical cache key is the kind, file and parameters, and an observation's identity additionally covers the file digest, the procedure and the result. Handled logical probes survive resets, so changing only a prediction does not cause a probe to run again. Assessment records contain the hypothesis and observation identifiers but no prediction or probe identifier; recovering the exact prediction requires the ordered proposal artifacts in the audit log.

A.5. Probe Ordering

For a probe p of hypothesis h_p, let $\alpha(p)$ count the other live hypotheses that, when p was proposed, had a prediction whose predicate or expected value differs from p's prediction. Probes in a round run in decreasing order of

$$U(p) = \frac{q(p)\Big(\max\{1, \alpha(p)\} + \sum_{h' \in H \setminus \{h_p\}} D(h_p, h')\Big)}{\max\{10^{-3}, c(p)\}}, \tag{6}$$

with ties broken by identifier. The reliability weight $q(p)$ is 0.9 for a source quote and 0.7 otherwise, and the estimated cost $c(p)$ is 1 for static probes and 30 for tests, traces and counterfactuals. Both are chosen ordering parameters. Because $\alpha(p)$ is captured as proposals are processed, an earlier proposal does not count an alternative introduced later in the same response. Probes stopped by the probe or time ceiling are not marked as handled.

A.6. Assessment Rules

For `source_contains`, the observed value is the result's `matched` field, whatever the probe kind, and the weight is 0.25; an AST probe paired with this predicate therefore receives source-presence scope. For `ast_matches` on an AST query, the value is `matched` and the weight is 1. For `exit_code`, `test_fails` and `output_contains` on a test, trace or counterfactual, and for `counterfactual_changes_outcome` on a counterfactual, the value comes from the procedure's return code or output, and the weight is 2. Any other combination is inconclusive. AST matching compares against the unparsed node text, which normalizes quotes and spacing.

A.7. Reset Conditions and Variants

Besides the conditions of Eq. (4), the monitor requires that the window's token total reach the reserve κ_0 and, optionally, that the consumed budget fraction $1 - B_{\text{rem}}/B_{\text{tot}}$ reach a threshold ϕ. With the default $\phi = 0$, the second

condition always holds. The repetition fraction u_t divides by the number of the round's validly grounded probes, and duplicates within one response do not count against each other. The fixed-reset variant resets after every third attempt, provided that attempt was accepted, and applies none of the monitor's conditions.

Table 14 checks the reset conditions on the running example with the library defaults. Four accepted rounds each consume 512 tokens. The first yields two observations, and the next three yield none and propose only executed checks, so each recent sample is $(g_t, u_t, d_t, \kappa_t) = (0, 1, 1, 512)$. The 16,000-token budget is the library default, separate from the study's 32,000-token ceiling.

Table 14
Constructed reset calculation for the running example.

Condition or quantity	Value and check
Accepted rounds, window	$n = 4 \geq n_0 = 4$; $\lvert\Omega\rvert = 3 = W$
Budget total, remaining	$B_{\text{tot}} = 16{,}000$; $B_{\text{rem}} = 13{,}952$
Consumed fraction	$0.128 \geq \phi = 0$
Window tokens	$3 \times 512 = 1{,}536 \geq \kappa_0 = 256$
Mean gain	$\bar{g} = 0 \leq \theta_g = 0$
Repetition or diversity	$\bar{u} = 1 \geq 0.5$ holds; $\bar{d} = 1 < 0.5$ fails
Restart estimate	$\max(256, \lfloor 512/2 \rfloor) = 256$
Affordability	$13{,}952 > 256 + 256$

The library also supports an independent-role configuration, which the study does not evaluate. In it, each role has its own hypothesis pool, behavior model and accessible observations, while the probe cache is shared. A reset prunes every pool and clears every dialogue. Handled-probe filtering is disabled in this configuration, one progress window collects the samples of all roles, and an observation reused from another role's check counts as new for the receiving role's progress sample.

A.8. Termination and Output

A new attempt requires at least 64 remaining tokens, and the probe and time ceilings are tested before each attempt and between probes. The controller does not impose a hard deadline on an LLM call. The execution-based stop requires support for the leading hypothesis from at least two distinct registered procedure identifiers, nonempty assumptions and no contradiction. It does not filter by assessment scope, and because it inspects the pool as it was before a reset in the same round, it sees hypotheses that the reset has just pruned, although the leading hypothesis itself is never pruned. The output's abstention flag is cleared by any supporting execution-based assessment in the run, even for a pruned or non-leading hypothesis, and the confidence field is always empty. Neither the stop nor the flag certifies the leading location. In the `no_probes` variant, the j-th proposal of a response receives the score $\max(\text{score}, 0.01(4 - j))$ for $j = 0, \ldots, 3$. A function ranking keeps the first occurrence of each symbol and may contain class-only entries, which exact-function scoring must keep apart from functions. Artifact digests and a hash-linked audit log provide integrity and provenance, not a proof of diagnostic correctness.

A.9. Library Defaults and Configuration Scope

Table 15 lists the library defaults. The study settings of Section 4.4 override the ceilings; all other values apply as listed. The library also provides controls outside this article, including `context_only`, `single_hypothesis`, `no_mde` and `independent_agents`; their implementation is not performance evidence.

B. Evidence and Reproduction Map

All empirical tables derive from the fixed study under `experiments/pilot/results/analysis/`; paths keep their original collection identifiers. The inputs are `scores.csv`, `summary.json`, `controls.json`, `quality.json`, `completeness.json` and `final-verification.json`, and the selection and protocol identity is recorded in `experiments/pilot/results/pilot.json`. Collection and reuse are documented in `experiments/pilot/README.md`. Together these specify the fifty issue identifiers, the 400 investigation scores and the 100 controls. Earlier benchmark aggregates and other cohorts are not inputs to the tables of this paper.

Table 15
Library defaults. All values are chosen, not tuned or calibrated; study settings override some of them.

Parameter	Default	Role	Module
Attempt / token / probe / time ceilings	12 / 16,000 / 48 / 3,600 s	Cumulative limits	config
Portfolio bound m / pool size / files per request	3 / 20 / 4	Visible hypotheses and files	config
Seed files / neighborhood hops	5 / 2	Candidate preparation	controller, pipeline
Excerpt lines / characters / relations	40 / 4,500 / 12	Request excerpts	config, controller
Proposals per response / admission floor	4 / 64 tokens	Proposal processing; loop admission	controller
W, n_0, κ_0, ϕ	3, 4, 256, 0	Window, rounds, reserve, consumed fraction	config
θ_g, θ_u, θ_d	0, 0.5, 0.5	Gain, repetition, diversity thresholds	config
Fixed reset interval	3 attempts	`fixed_reset` variant	config
Dissimilarity weights (words / file)	0.6 / 0.4	Eq. (2)	policies
Probe reliability q (source quote / other)	0.9 / 0.7	Eq. (6)	pipeline
Probe cost c (static / execution)	1 / 30; floor 10^{-3}	Eq. (6)	pipeline, policies
Assessment weight w	0.25 / 1 / 2	Eq. (1); inconclusive 0	pipeline
`no_probes` proposal scores	$0.01(4-j)$, $j = 0, \ldots, 3$	Proposal-order scores	controller

From the repository root, `docs/JSS/scripts/analyze_completed_study.mjs` checks every model, workflow, issue and seed key, eligibility, tokens, saved means, controls, reset exposure and paired hit and pool changes. With `-write` it emits the six result tables under `docs/JSS/generated/`, which are reproduced verbatim in this paper; otherwise it checks their deterministic reconstruction. It writes `docs/JSS/validation/completed-study-audit.json` with input digests. This is a reconstruction of the score tables, not a fresh validation of raw outputs and signatures, and the original cluster intervals remain tied to their saved report and stated procedure.

The frozen protocol SHA-256 is `3afba2e46e120a3e35fc585e16ab9ddd474605ec93a5294abd8c618ad740f076`. Model and dataset pins are in `experiments/full/pins.json`. The checkpoint commits are `c03e6d358207e414f1eca0bb1891e29f1db0e242` (Qwen) and `e5d64addd26a6a1db0f9b863abf6ee3141936807` (DeepSeek), and the dataset commits are `6ec7bb89b9342f664a54a6e0a6ea6501d3437cc2` (Lite) and `c104f840cc67f8b6eec6f759ebc8b2693d585d4a` (Verified). Large raw archives must be deposited or durably linked. **[AUTHOR TO PROVIDE: archive exact frozen source/environment, raw predictions and permissible traces, receipts, labels and clean-environment reproduction instructions.]**

Data Availability

The working repository identifies `https://github.com/ai4se4ai-lab/MORPH-FL` as the project location. Public accessibility and completeness of the evaluated-source and result archive have not been independently established in this revision. Local evidence and analysis are mapped in Appendix B. **[AUTHOR TO PROVIDE: provide the accessible replication-package URL and archival DOI, immutable evaluated code/protocol version, raw predictions and permissible audit traces, label-generation inputs, dependency lock and licenses.]** No completed open archival deposit is claimed.

CRediT authorship contribution statement

[AUTHOR TO PROVIDE: confirm the full author list and each author's applicable CRediT roles.]

Declaration of competing interests

[AUTHOR TO PROVIDE: provide the verified competing-interest declaration or confirm no competing interests.]

Funding

[AUTHOR TO PROVIDE: provide funder names, grant numbers and sponsor roles, or confirm no specific funding.]

Declaration of generative AI and AI-assisted technologies in the writing process

Generative AI tools were used to assist with organizing the repository code and checking the manuscript for grammatical errors. The author reviewed the generated suggestions and takes full responsibility for the accuracy and integrity of all claims, proofs, references, and the final manuscript.

References

Abreu, R., Zoeteweij, P., van Gemund, A.J.C., 2007. On the accuracy of spectrum-based fault localization, in: Testing: Academic and Industrial Conference Practice and Research Techniques – MUTATION (TAICPART-MUTATION), IEEE. pp. 89–98. doi:10.1109/TAIC.PART.2007.13.

Aleithan, R., Xue, H., Mohajer, M.M., Nnorom, E., Uddin, G., Wang, S., 2024. SWE-bench+: Enhanced coding benchmark for LLMs. arXiv:2410.06992. URL: https://arxiv.org/abs/2410.06992, doi:10.48550/arXiv.2410.06992.

Arora, D., Sonwane, A., Wadhwa, N., Mehrotra, A., Utpala, S., Bairi, R., Kanade, A., Natarajan, N., 2024. MASAI: Modular architecture for software-engineering AI agents. arXiv:2406.11638. URL: https://arxiv.org/abs/2406.11638, doi:10.48550/arXiv.2406.11638.

Badertdinov, I., Golubev, A., Nekrashevich, M., Shevtsov, A., Karasik, S., Andriushchenko, A., Trofimova, M., Litvintseva, D., Yangel, B., 2025. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents, in: Advances in Neural Information Processing Systems 38 (NeurIPS), Datasets and Benchmarks Track. URL: https://arxiv.org/abs/2505.20411. arXiv:2505.20411.

Blair, G., Bencomo, N., France, R.B., 2009. Models@run.time. Computer 42, 22–27. doi:10.1109/MC.2009.326.

Bouzenia, I., Devanbu, P., Pradel, M., 2025. RepairAgent: An autonomous, LLM-based agent for program repair, in: Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE), IEEE. pp. 2188–2200. doi:10.1109/ICSE55347.2025.00157.

Cai, B., Zou, W., Meng, Q., Xu, H., Zhang, J., 2025. KBL: A golden keywords-based query reformulation approach for bug localization. Empirical Software Engineering 30, 135. doi:10.1007/s10664-025-10694-2.

Carbonell, J., Goldstein, J., 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries, in: Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, ACM. pp. 335–336. doi:10.1145/290941.291025.

Chang, X., Li, D., Wong, W.E., 2025. Large language models for software fault localization: A survey, in: Proceedings of the 12th International Conference on Dependable Systems and Their Applications (DSA), IEEE. pp. 111–120. doi:10.1109/DSA66321.2025.00024.

Chen, Z., Tang, R., Deng, G., Wu, F., Wu, J., Jiang, Z., Prasanna, V., Cohan, A., Wang, X., 2025. LocAgent: Graph-guided LLM agents for code localization, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria. pp. 8697–8727. doi:10.18653/v1/2025.acl-long.426.

Chowdhury, N., Aung, J., Shern, C.J., Jaffe, O., Sherburn, D., Starace, G., Mays, E., Dias, R., Aljubeh, M., Glaese, M., Jimenez, C.E., Yang, J., Ho, L., Patwardhan, T., Liu, K., Madry, A., 2024. Introducing SWE-bench Verified. OpenAI Blog. URL: https://openai.com/index/introducing-swe-bench-verified/.

Efron, B., Tibshirani, R.J., 1993. An Introduction to the Bootstrap. Springer US. doi:10.1007/978-1-4899-4541-9.

Erman, L.D., Hayes-Roth, F., Lesser, V.R., Reddy, D.R., 1980. The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. ACM Computing Surveys 12, 213–253. doi:10.1145/356810.356816.

Field, C.A., Welsh, A.H., 2007. Bootstrapping clustered data. Journal of the Royal Statistical Society: Series B (Statistical Methodology) 69, 369–390. doi:10.1111/j.1467-9868.2007.00593.x.

Gomes, C.P., Selman, B., Kautz, H., 1998. Boosting combinatorial search through randomization, in: Proceedings of the 15th National Conference on Artificial Intelligence (AAAI), AAAI Press. pp. 431–437.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y.K., Luo, F., Xiong, Y., Liang, W., 2024. DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence. arXiv:2401.14196. URL: https://arxiv.org/abs/2401.14196, doi:10.48550/arXiv.2401.14196.

Hong, K., Troynikov, A., Huber, J., 2025. Context Rot: How Increasing Input Tokens Impacts LLM Performance. Technical Report. Chroma. URL: https://research.trychroma.com/context-rot.

Hu, M., Chen, T., Chen, Q., Mu, Y., Shao, W., Luo, P., 2025. HiAgent: Hierarchical working memory management for solving long-horizon agent tasks with large language model, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria. pp. 32779–32798. doi:10.18653/v1/2025.acl-long.1575.

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., Dang, K., Fan, Y., Zhang, Y., Yang, A., Men, R., Huang, F., Zheng, B., Miao, Y., Quan, S., Feng, Y., Ren, X., Ren, X., Zhou, J., Lin, J., 2024. Qwen2.5-Coder technical report. arXiv:2409.12186. URL: https://arxiv.org/abs/2409.12186, doi:10.48550/arXiv.2409.12186.

Jiang, H., Wu, Q., Lin, C.Y., Yang, Y., Qiu, L., 2023. LLMLingua: Compressing prompts for accelerated inference of large language models, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics. pp. 13358–13376. doi:10.18653/v1/2023.emnlp-main.825.

Jiang, Z., Ren, X., Yan, M., Jiang, W., Li, Y., Liu, Z., 2025. Issue localization via LLM-driven iterative code graph searching, in: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), IEEE. pp. 3034–3045. doi:10.1109/ASE63991.2025.00249.

Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K., 2024. Swe-bench: Can language models resolve real-world github issues?, in: International Conference on Learning Representations, pp. 54107–54157.

Jones, J.A., Harrold, M.J., 2005. Empirical evaluation of the Tarantula automatic fault-localization technique, in: Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering (ASE), ACM. pp. 273–282. doi:10.1145/1101908.1101949.

Kang, M., Chen, W.N., Han, D., Inan, H.A., Wutschitz, L., Chen, Y., Sim, R., Rajmohan, S., 2026. ACON: Optimizing context compression for long-horizon LLM agents, in: Proceedings of the 43rd International Conference on Machine Learning (ICML). URL: https://arxiv.org/abs/2510.00615. arXiv:2510.00615.

Kang, S., An, G., Yoo, S., 2024. A quantitative and qualitative evaluation of LLM-based explainable fault localization. Proceedings of the ACM on Software Engineering 1, 1424–1446. doi:10.1145/3660771.

Kang, S., Chen, B., Yoo, S., Lou, J.G., 2025. Explainable automated debugging via large language model-driven scientific debugging. Empirical Software Engineering 30, 45. doi:10.1007/s10664-024-10594-x.

Kephart, J.O., Chess, D.M., 2003. The vision of autonomic computing. Computer 36, 41–50. doi:10.1109/MC.2003.1160055.

de Kleer, J., 1986. An assumption-based TMS. Artificial Intelligence 28, 127–162. doi:10.1016/0004-3702(86)90080-9.

de Kleer, J., Williams, B.C., 1987. Diagnosing multiple faults. Artificial Intelligence 32, 97–130. doi:10.1016/0004-3702(87)90063-4.

Laban, P., Hayashi, H., Zhou, Y., Neville, J., 2026. LLMs get lost in multi-turn conversation, in: Proceedings of the 14th International Conference on Learning Representations (ICLR). URL: https://arxiv.org/abs/2505.06120. arXiv:2505.06120.

Li, Y., Wang, S., Nguyen, T.N., 2021. Fault localization with code coverage representation learning, in: Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE), IEEE. pp. 661–673. doi:10.1109/ICSE43902.2021.00067.

Liang, S., Garg, S., Zilouchian Moghaddam, R., 2026. The SWE-Bench illusion: When state-of-the-art LLMs remember instead of reason, in: Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice, ACM. pp. 395–405. URL: https://doi.org/10.1145/3786583.3786882, doi:10.1145/3786583.3786882. arXiv:2506.12286.

Lindenbauer, T., Slinko, I., Felder, L., Bogomolov, E., Zharov, Y., 2025. The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management, in: Proceedings of the 4th Deep Learning for Code (DL4C) Workshop at NeurIPS. URL: https://arxiv.org/abs/2508.21433. arXiv:2508.21433.

Liu, B., Lucia, Nejati, S., Briand, L.C., Bruckmann, T., 2016. Simulink fault localization: An iterative statistical debugging approach. Software Testing, Verification and Reliability 26, 431–459. doi:10.1002/stvr.1605.

Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P., 2024. Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics 12, 157–173. doi:10.1162/tacl_a_00638.

Luo, Z., Guo, J., He, W., Wang, S., Yu, J., Zhao, F., Chen, Y., Cao, T., Liu, L., Zheng, N., Xu, W., Jiang, J., Zhao, Z., 2026. AdaRepair-Mem: Adaptive experience orchestration for repository-level program repair. arXiv:2609.20130. URL: https://arxiv.org/abs/2609.20130, doi:10.48550/arXiv.2609.20130.

Moreau, L., Missier, P., 2013. PROV-DM: The PROV data model. W3C Recommendation. URL: https://www.w3.org/TR/2013/REC-prov-dm-20130430/. luc Moreau and Paolo Missier, Eds.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S.G., Stoica, I., Gonzalez, J.E., 2024. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560v2. URL: https://arxiv.org/abs/2310.08560v2, doi:10.48550/arXiv.2310.08560.

Park, J.S., O'Brien, J., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S., 2023. Generative agents: Interactive simulacra of human behavior, in: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST), ACM. pp. 1–22. doi:10.1145/3586183.3606763.

Qin, Y., Wang, S., Lou, Y., Dong, J., Wang, K., Li, X., Mao, X., 2025. SoapFL: A standard operating procedure for LLM-based method-level fault localization. IEEE Transactions on Software Engineering 51, 1173–1187. doi:10.1109/TSE.2025.3543187.

Rafi, M.N., Kim, D.J., Chen, T.H., Wang, S., 2024. A multi-agent approach to fault localization via graph-based retrieval and reflexion. arXiv:2409.13642. URL: https://arxiv.org/abs/2409.13642, doi:10.48550/arXiv.2409.13642.

Reddy, R.G., Suresh, T., Doo, J., Liu, Y., Nguyen, X.P., Zhou, Y., Yavuz, S., Xiong, C., Ji, H., Joty, S., 2026. SweRank: Software issue localization with code ranking, in: Proceedings of the 14th International Conference on Learning Representations (ICLR). URL: https://arxiv.org/abs/2505.07849. arXiv:2505.07849.

Reiter, R., 1987. A theory of diagnosis from first principles. Artificial Intelligence 32, 57–95. doi:10.1016/0004-3702(87)90062-2.

Robertson, S., Zaragoza, H., 2009. The probabilistic relevance framework: Bm25 and beyond. Foundations and trends® in information retrieval 4, 1–174.

Schmidt, D.C., 2006. Guest editor's introduction: Model-driven engineering. Computer 39, 25–31. doi:10.1109/MC.2006.58.

Seddik, S., Seddik, F., Esmaeili, A., Sadatbenis, M., Fard, F., 2026. ARISE: A repository-level graph representation and toolset for agentic program repair and fault localization. arXiv:2605.03117v2. URL: https://arxiv.org/abs/2605.03117, doi:10.48550/arXiv.2605.03117.

Sepidband, M., Taherkhani, H., Pham, H.V., Hemmati, H., 2026. RGFL: Reasoning guided fault localization for automated program repair using large language models. arXiv:2601.18044. URL: https://arxiv.org/abs/2601.18044, doi:10.48550/arXiv.2601.18044.

Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E.H., Schärli, N., Zhou, D., 2023. Large language models can be easily distracted by irrelevant context, in: Proceedings of the 40th International Conference on Machine Learning (ICML), pp. 31210–31227. URL: https://proceedings.mlr.press/v202/shi23a.html.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S., 2023. Reflexion: Language agents with verbal reinforcement learning, in: Advances in Neural Information Processing Systems 36 (NeurIPS), pp. 8634–8652. doi:10.52202/075280-0377.

Tam, Z.R., Wu, C.K., Tsai, Y.L., Lin, C.Y., Lee, H.y., Chen, Y.N., 2024. Let me speak freely? A study on the impact of format restrictions on large language model performance, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, Association for Computational Linguistics, Miami, FL, USA. pp. 1218–1236. doi:10.18653/v1/2024.emnlp-industry.91.

Tan, S., Zhou, B., Agrawal, L.A., Zhang, J., Chen, Y., Mang, Q., Lee, D., Jacob, D., Sen, K., Dimakis, A., Zaharia, M., 2026. Scaling discovery requires forgetting. arXiv:2610.20563. URL: https://arxiv.org/abs/2610.20563, doi:10.48550/arXiv.2610.20563.

Troya, J., Segura, S., Parejo, J.A., Ruiz-Cortés, A., 2018. Spectrum-based fault localization in model transformations. ACM Transactions on Software Engineering and Methodology 27, 13:1–13:50. doi:10.1145/3241744.

Wang, B., Xu, W., Li, Y., Gao, M., Xie, Y., Sun, H., Chen, D., 2026a. Improving code localization with repository memory, in: Proceedings of the 14th International Conference on Learning Representations (ICLR). URL: https://openreview.net/forum?id=8yjWLJy2eX.

Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., Ji, H., 2024. Executable code actions elicit better LLM agents, in: Proceedings of the 41st International Conference on Machine Learning (ICML), pp. 50208–50232. URL: https://proceedings.mlr.press/v235/wang24h.html.

Wang, X., Li, B., Song, Y., Xu, F.F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., Tran, H.H., Li, F., Ma, R., Zheng, M., Qian, B., Shao, Y., Muennighoff, N., Zhang, Y., Hui, B., Lin, J., Brennan, R., Peng, H., Ji, H., Neubig, G., 2025. OpenHands: An open platform for AI software developers as generalist agents, in: Proceedings of the 13th International Conference on Learning Representations (ICLR). URL: https://arxiv.org/abs/2407.16741. arXiv:2407.16741.

Wang, Z., Xu, Y., Li, C., Peng, C., Adams, B., Hassan, A.E., Chen, T.H.P., 2026b. Turning interaction history into execution state: A runtime layer for long-horizon coding agents. arXiv:2608.00808. URL: https://arxiv.org/abs/2608.00808, doi:10.48550/arXiv.2608.00808.

Wong, W.E., Gao, R., Li, Y., Abreu, R., Wotawa, F., 2016. A survey on software fault localization. IEEE Transactions on Software Engineering 42, 707–740. doi:10.1109/TSE.2016.2521368.

Wu, L., Wei, J., Wu, Y., Du, B., Zhao, R., Liu, Y., 2026. Share-rag: A structure-aware and hypothesis-guided retrieval-augmented framework for fault localization in novice programs. Journal of Systems and Software , 113057.

Wu, Y., Liu, Y., Yin, Y., Zeng, M., Ye, Z., Zhang, X., Xiong, Y., Zhang, L., 2025. SmartFL: Semantics based probabilistic fault localization. IEEE Transactions on Software Engineering 51, 2161–2180. doi:10.1109/TSE.2025.3574487.

Xia, C.S., Deng, Y., Dunn, S., Zhang, L., 2025. Demystifying LLM-based software engineering agents. Proceedings of the ACM on Software Engineering 2, 801–824. doi:10.1145/3715754.

Xu, C., Liu, Z., Ren, X., Zhang, G., Liang, M., Lo, D., 2025a. FlexFL: Flexible and effective fault localization with open-source large language models. IEEE Transactions on Software Engineering 51, 1455–1471. doi:10.1109/TSE.2025.3553363.

Xu, H., Liu, H., Wu, Y., Kang, X., Chen, X., Liu, Y., 2026. Exploring the potential and limitations of large language models for novice program fault localization. Journal of Systems and Software 234, 112731. doi:10.1016/j.jss.2025.112731.

Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., Zhang, Y., 2025b. A-Mem: Agentic memory for LLM agents, in: Advances in Neural Information Processing Systems 38 (NeurIPS). URL: https://arxiv.org/abs/2502.12110. arXiv:2502.12110.

Yang, A.Z.H., Le Goues, C., Martins, R., Hellendoorn, V.J., 2024a. Large language models for test-free fault localization, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), ACM. pp. 1–12. doi:10.1145/3597503.3623342.

Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O., 2024b. SWE-agent: Agent-computer interfaces enable automated software engineering, in: Advances in Neural Information Processing Systems 37 (NeurIPS), pp. 50528–50652. doi:10.52202/079017-1601.

Yeo, I., Ryu, D., Baik, J., 2025. Improving LLM-based fault localization with external memory and project context. arXiv:2506.03585. URL: https://arxiv.org/abs/2506.03585, doi:10.48550/arXiv.2506.03585.

Yu, Z., Zhang, H., Zhao, Y., Huang, H., Yao, M., Ding, K., Zhao, J., 2025. OrcaLoca: An LLM agent framework for software issue localization, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), pp. 73416–73436. URL: https://proceedings.mlr.press/v267/yu25x.html.

Zeller, A., 2009. Why Programs Fail: A Guide to Systematic Debugging. 2 ed., Morgan Kaufmann, Burlington, MA, USA. URL: https://www.whyprogramsfail.com/.

Zhang, Y., Ruan, H., Fan, Z., Roychoudhury, A., 2024. AutoCodeRover: Autonomous program improvement, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), ACM. pp. 1592–1604. doi:10.1145/3650212.3680384.

Zhong, W., Guo, L., Gao, Q., Ye, H., Wang, Y., 2024. MemoryBank: Enhancing large language models with long-term memory, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 19724–19731. doi:10.1609/aaai.v38i17.29946.

Zhou, J., Zhang, H., Lo, D., 2012. Where should the bugs be fixed? more accurate information retrieval-based bug localization based on bug reports, in: Proceedings of the 34th International Conference on Software Engineering (ICSE), IEEE. pp. 14–24. doi:10.1109/ICSE.2012.6227210.